INFORMATION MANAGEMENT

OBJECT-ORIENTED PROGRAMMING

PROJECT

Project/System Title: Transparent and Centralized Grading Management System

Group Members: Oropilla, Condes, Lalata, Mariano, Puyat

Course / Section: BSCS - 2A

Professor: Mrs. Rebecca R. Fajardo

Project Documentation and Submission Map

Stage	Required Output	Primary Evidence	Professor Remarks
1	Concept Paper	Problem, objectives, scope, users, proposed solution	☐
2	Business Rules	Operational rules, constraints, data relationships	☐
3	ERD	Entities, attributes, PK/FK, cardinality, optionality	☐
4	Normalization	UNF/initial data → 1NF → 2NF → 3NF	☐
5	Data Dictionary	Complete field definitions and constraints	☐
6	MySQL Database	DDL, DML, sample records, queries, screenshots	☐
7	VB.NET System	Forms, CRUD, validation, database connectivity	☐
8	Test Cases	Functional tests, expected/actual results, status	☐
9	User Manual	Installation/access and step-by-step usage guide	☐

General Documentation Standards

☐ All entities and fields shown in the ERD must correspond to the normalized tables and data dictionary.

☐ Primary keys, foreign keys, data types, constraints, and relationships must match the implemented MySQL database.

☐ VB.NET forms and functions must use the actual MySQL database designed in this project.

☐ Use screenshots only as supporting evidence; explain what each screenshot demonstrates.

☐ Use meaningful sample data. Do not use sensitive real-world personal data unless authorized and properly anonymized.

☐ Number and caption major figures/tables (e.g., Figure 1. ERD; Table 3. Data Dictionary).

☐ Attach source code/database scripts as appendices or as separate electronic files when required by the instructor.

1. Concept Paper

The concept paper establishes the need for the proposed system and defines what the group intends to develop.

1.1 Project Title
State a concise and specific title for the proposed database-driven system.

Transparent and Centralized Grading Management System

1.2 Background / Rationale
In academic institutions, traditional grade computation and dispute resolution often rely on paper records or isolated spreadsheets. This decentralization creates significant delays in providing students with real-time feedback on their running marks and weighting distributions. Furthermore, handling grade disputes verbally or informally lacks verifiable tracking, which can lead to administrative mistrust, human error, and loss of records. A centralized, database-driven system bridges these gaps by enforcing real-time calculation rules, automating grade change audits, and providing clear role-based portals for students, faculty, and administrators

1.3 Problem Statement
Identify the main problem and supporting problems that the system will address.

Primary Problem: Inefficiencies, vulnerability to calculation errors, and a lack of real-time transparency in traditional manual grade recording and dispute resolution processes.

Supporting Problems:

1. Delays in student visibility regarding individual assessment marks, weighting distributions, and running grades.
2. Absence of an auditable tracking mechanism for grade modifications and formal revision requests.
3. Administrative friction in managing user credentials, class section enrollments, and faculty assignments across academic departments.

1.4 General Objective
State the overall purpose of the project.

To design, implement, and deploy a secure, database-driven grading management system that provides real-time running grade computation, transparent dispute tracking, and auditable record management for Pamantasan ng Lungsod ng Pasig

1.5 Specific Objectives
List measurable objectives. Include database-related and system-related objectives.

Database Objectives: Implement a 3NF-normalized MariaDB/MySQL relational schema with parameterized queries, referential integrity constraints, and SHA-256 hashed password storage.

System Objectives: Develop a VB.NET Windows Forms application with role-based access control tailored for Students, Teachers, and System Administrators.

Functional Objectives:

- Automate real-time running grade calculations based on dynamic category weights (Quizzes, Activities, Exams, Projects).
- Build a digital revision request workflow where students submit score disputes and teachers review, approve, or reject changes with mandatory reason logging.
- Implement automated system audit logs capturing all grade modifications with timestamps and user attributions.

Output Objectives: Generate downloadable PDF and CSV class grade sheets for official administrative reporting.

1.6 Proposed Solution / System Overview

Explain how the proposed system will work and how users will interact with it.

The proposed platform establishes a single source of truth for academic performance. Teachers record raw assessment scores, which dynamically trigger weight calculations and update student status indicators ("Passed" / "At Risk"). Students gain transparent access to their live running average, score history, and assessment breakdowns. When discrepancies occur, students submit a formal digital Revision Request. Teachers evaluate requests within their portal, and any approved change automatically updates the database and writes an immutable record to the Audit Log. Administrators maintain overarching control through user profile management and class allocation directories

1.7 Target Users / Stakeholders

Identify the intended users and their roles.

User	Role in the system	Main features they use
Students	View their own grades and check how they are doing	Live grade report with running grade and standing, grade history showing who changed a grade and why, revision requests with status tracking, and a performance overview
Professors	Record and manage grades for their classes	Gradebook for entering scores, handling student revision requests, and viewing their assigned classes
Administrators	Manage the system and its records	Student and professor management (add, edit, search, filter), class and subject setup, request monitoring, and audit logs

1.8 Scope

State the functions, records, transactions, reports, and users included in the project.

1. Authentication and SHA-256 password validation.

2. Score entry and automated dynamic grade recalculation.
3. Digital class gradebook with PDF/CSV export functionality.
4. Student grade dispute submission and teacher approval/rejection panel.
5. Class performance analytics dashboard.
6. Comprehensive grade audit logging.
7. Admin user management (CRUD for Students, Professors, Classes).

1.9 Limitations

State functions or conditions intentionally excluded from the project.

1. No Cloud Connection
2. No Google Classroom Integration
3. Can't store images, Videos
4. No Automatic Online Backup
5. Local Database Only

1.10 Significance / Expected Benefits

Explain the expected benefits to users, organization, and information management process.

To Students: Delivers immediate feedback on academic standing, eliminating surprise failing marks and ensuring fair dispute resolution.

To Faculty: Saves manual calculation time, eliminates human arithmetic errors, and provides structured documentation for grade adjustments.

To Administration: Enhances institutional integrity, guarantees full auditable accountability, and streamlines user directory management.

1.11 Major System Modules

Module	Main Functions	Primary User/s
Authentication & Session	Role-based login, password verification, active App Session management.	All Users
Teacher Gradebook & Entry	Raw score recording, running grade recalculation, gradebook filter, PDF/CSV export.	Professors
Revision Module	Dispute submission, request queue management, status updates	Students and Professors

	(Pending/Approved/Closed).	
Student/Professor Dashboard	Live grade breakdown, percentage weighting display, score revision history.	Students for student dashboard & Professors for professor dashboard
Analytics & Audit Log	Class averages, top scores, at-risk flags, timestamped audit log of all changes.	All users
Admin Management Portal	Student/Professor directory CRUD, class assignments, system stats.	Admin

2. Business Rules

Business rules describe how the organization operates and how data must be stored, related, validated, and processed. Rules should be clear enough to guide the ERD and database constraints.

2.1 Narrative Business Rules

Write at least the rules needed to define users, records, transactions, relationships, validation, and status changes. Use statements such as “One customer may have many orders” or “Each equipment record must belong to one category.”

Rule No.	Business Rule	Database/Relationship Impact	Validation/Constraint
1	Each user must possess a unique account with an assigned role (Admin, Teacher, Student).	Users table primary key (user_id), unique constraint on username / student_number.	NOT NULL, Hashed SHA-256 password requirement
2	A student can be enrolled in multiple classes; each class contains multiple students.	Resolves M:N relationship using Enrollments junction table.	Foreign Keys referencing Students and Classes.
3	Assessment scores must not exceed the defined maximum score for that item.	Enforced in Scores table.	CHECK (score >= 0 AND score <= max_points).
4	Running grades are calculated dynamically based on predefined assessment weightings.	Computed via application business logic / database views.	Weights across active categories must sum to 100%.

5	A student may submit a revision request only for existing, published scores	Foreign Key in RevisionRequests referencing score_id.	Single pending request per assessment item constraint
6	Approving a revision request automatically updates the score and records an audit entry.	Database transaction updating Scores and inserting into AuditLogs.	Status constraint: Pending, Approved, Under Review, Closed.
7	Every grade insertion, edit, or dispute approval must produce a timestamped log entry.	Insert-only AuditLogs table.	Foreign Key referencing changed_by_user_id, TIMESTAMP DEFAULT CURRENT_TIMESTAMP.
8	Only active professors assigned to a specific course can modify scores or export grade sheets.	Foreign Key in Classes referencing professor_id.	Application-level authorization check via AppSession.
9			
10			

2.2 Assumptions

1. One academic year is divided into distinct terms (e.g., Midterms, Finals).

2. Category weights (Quizzes: 10%, Activities: 15%, Midterms: 25%, Projects: 20%, Finals: 30%) remain uniform within a given course module unless updated by administration.

3. Approved score adjustments take effect immediately across all student and teacher dashboard views

2.3 User Roles and Permissions

User Role	Can View	Can Add/Edit/Delete	Special Permissions
Admin	System Dashboard, Student Directory, Professor Directory, Global Audit Logs.	Add/Edit/Deactivate Students, Professors, and Class Assignments.	Full System Administration
Student	Own Live Grades, Grade	Submit Revision	Export Grade Sheets to

	History, Performance Stats, Own Revision Requests.	Requests. PDF	PDF/CSV.
Teacher	Assigned Class Gradebooks, Revision Request Queue, Class Analytics, Grade Audit Logs.	Add/Edit Assessment Scores, Approve/Reject Revision Requests.	Full System Administration

3. Entity-Relationship Diagram (ERD)

Create the ERD based on the approved business rules. The final ERD must show entities, attributes, primary keys, foreign keys, relationships, cardinality, and optionality where applicable.

☐ All required entities are present.

☐ Every entity has a primary key.

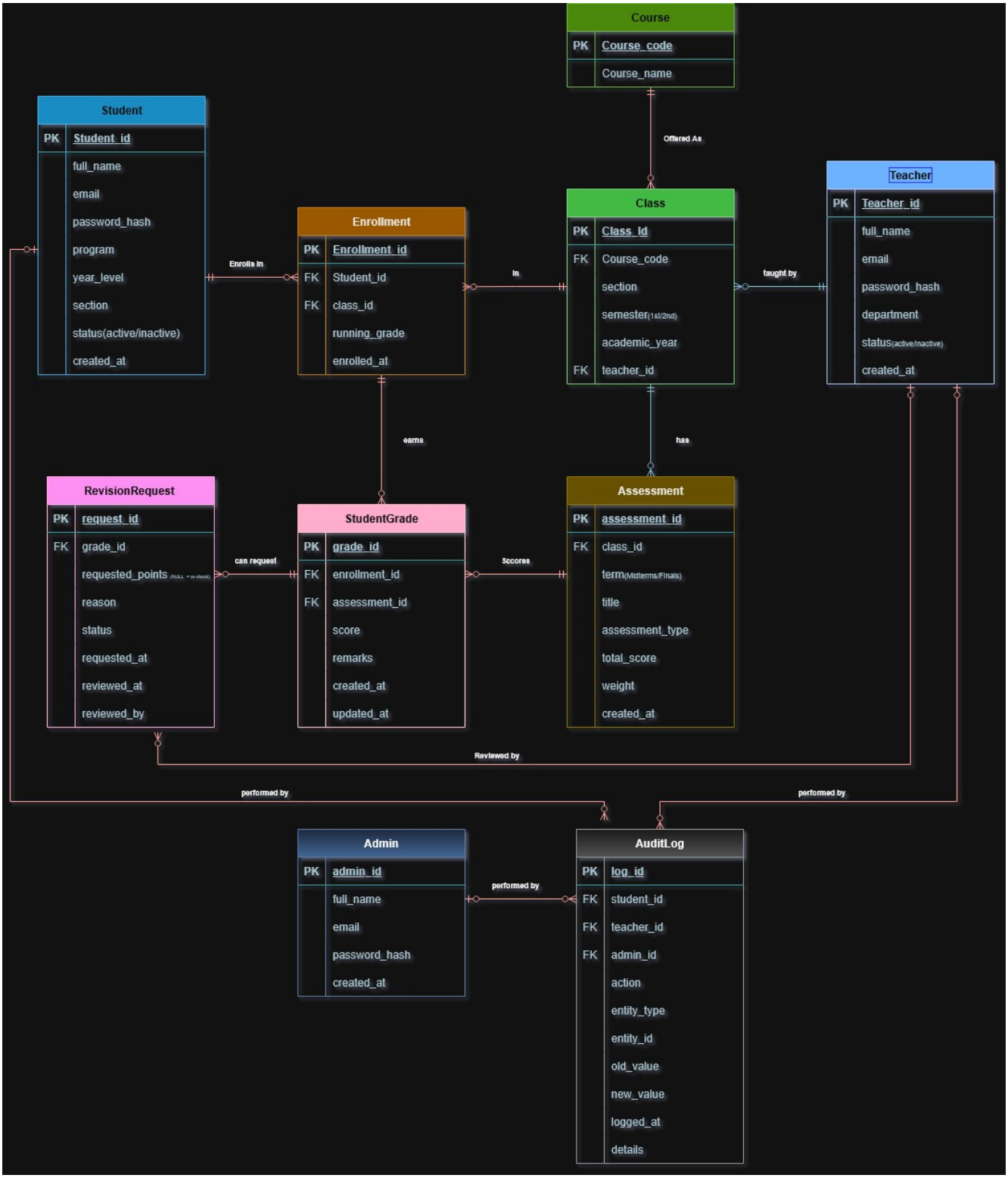
☐ Foreign keys correctly represent relationships.

☐ Many-to-many relationships are resolved using associative/intersection entities.

☐ Attribute names are meaningful and consistent.

☐ Cardinality/optionality agree with the business rules.

3.1 ERD



3.2 ERD Explanation

Student

Stores each student's account and details, such as name, email, program, year level and section. It matters because students are the people the system exists for. They log in to see their grades and to ask for a re-check. It connects to Enrollment, which shows the classes the student takes, and to AuditLog, which records the student's actions.

Teacher
Stores each teacher's account and department. It matters because teachers record the scores and review revision requests. It connects to Class, which shows the classes they handle, to RevisionRequest, which shows the requests they reviewed, and to AuditLog, which records their actions.

Admin
Stores the administrator accounts that manage the system, such as creating accounts and assigning classes. It matters because someone must be accountable for system setup. It connects to AuditLog, which records every admin action.

Course
Stores the subject itself, such as CS 103 and its name. It matters because the course name is stored once here instead of being repeated in every class. This keeps the data clean and follows third normal form. It connects to Class, since one course can be offered as many classes.

Class
Stores one offering of a course: the section, the semester and the academic year. It matters because grades and enrollments belong to a specific class, not just to a subject. It connects to Course (what is taught), Teacher (who teaches it), Enrollment (who is in it) and Assessment (what is graded in it).

Enrollment
Stores which student is in which class, with the running grade for that class. It matters because a student takes many classes and a class has many students. Without it we could not link them. It connects to Student, Class and StudentGrade, which holds the student's scores in that class.

Assessment
Stores each quiz, activity, exam or project, with its term (Midterm or Finals), total score and weight. It matters because it defines what is graded and how much each item counts. It connects to Class (which class it belongs to) and StudentGrade (the scores students got on it).

StudentGrade
Stores one student's score on one assessment, with remarks and timestamps. It matters because it holds the actual grades, which is the main data the system shows. It connects to Enrollment (whose grade it is), Assessment (what was graded) and RevisionRequest (any dispute about the score).

RevisionRequest
Stores a student's request to re-check a score, with the reason, the points asked for and the review status. It matters because it is what makes the system transparent: students can question a grade, and the outcome is recorded. It connects to StudentGrade (the grade in question) and Teacher (who reviewed it).

AuditLog
Stores a permanent record of who did what and when, including the old and new value of a changed score. It matters because it makes every change traceable, so no grade can be changed without a record. It connects to Student, Teacher and Admin, and each entry names exactly one of them. The record that changed is stored as plain text, so the history stays even if that record is deleted later.

4. Database Normalization: 1NF to 3NF

Show the transformation of the initial/un-normalized data structure into First, Second, and Third Normal Form. Do not submit only the final tables; demonstrate the reasoning at each stage.

4.1 Initial / Unnormalized Data

Present the original transaction, form, spreadsheet, or record structure. Identify repeating groups, multi-valued attributes, and redundancy.

GRADESHEET

student_id (PK)	student_name	student_email	student_password_hash	program	year_level	student_section	student_status	student_created_at	class_id
25-00418	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active		1 3
25-00731	Jerome Pagdanganan	jerome.pagdanganan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active		1 3

course_code	course_name	class_section	semester	academic_year	teacher_id	teacher_name	teacher_email	teacher_password_hash	department
CS 103 COMP 105	Object-Oriented Programming (VB.NET with Database) Information Management	A A	1st Semester 1st Semester	2026-2027 2026-2027	P001 P002	Jhun Alvarez Rebecca R. Fajardo	jhun.alvarez@dauntlesswolves.test rebecca.fajardo@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND REPLACE_WITH_HASH_FROM_BACKEND	CCS CCS
CS 103 COMP 105	Object-Oriented Programming (VB.NET with Database) Information Management	A A	1st Semester 1st Semester	2026-2027 2026-2027	P001 P002	Jhun Alvarez Rebecca R. Fajardo	jhun.alvarez@dauntlesswolves.test rebecca.fajardo@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND REPLACE_WITH_HASH_FROM_BACKEND	CCS CCS

eacher_status	teacher_created_at	enrollment_id	running_grade	enrolled_at	assessment_id	term	title	assessment_type
ctive		1	NULL		1	Midterm	Quiz 1	Quiz
ctive		21	NULL		2	Midterm	Activity 1	Activity
					3	Midterm	Midterm Exam	Exam
					4	Finals	Quiz 2	Quiz
					5	Finals	Project	Project
					6	Finals	Finals Exam	Exam
					---	---	---	---
					13	Midterm	Quiz 1	Quiz
					14	Midterm	Activity 1	Activity
					15	Midterm	Midterm Exam	Exam
					16	Finals	Quiz 2	Quiz
					17	Finals	Project	Project
					18	Finals	Finals Exam	Exam
ctive		2	NULL		1	Midterm	Quiz 1	Quiz
ctive		22	NULL		2	Midterm	Activity 1	Activity
					3	Midterm	Midterm Exam	Exam
					4	Finals	Quiz 2	Quiz

total_score	weight	assessment_created_at	grade_id	score	remarks	grade_created_at	grade_updated_at
20	10		1	14			
50	10		2	41			
100	30		3	82			
20	10		121	NULL			
100	15		122	NULL			
100	25		123	NULL			
---	---	---	---	---	---	---	---
20	10		61	13			
50	10		62	36			
100	30		63	69			

REVISIONREQUEST

request_id (PK)	grade_id	requested_points	reason	request_status	requested_at	reviewed_at	reviewed_by
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001

AUDITLOG

log_id (PK)	action	entity_type	entity_id	old_value	new_value	logged_at	details	actor_student_id	actor_teacher_id	admin_id
1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin			
2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001			2
3	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to P002			2
4	Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts	25-01894		

ADMIN

admin_id (PK)	admin_name	admin_email (AK)	admin_password_hash	admin_created_at	
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND		
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND		
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND		
4	Geoff Andrei Mariano	geoff.mariano@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND		
5	Ron Jacob Puyat	ron.puyat@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND		

4.2 First Normal Form (1NF)

Show atomic values, removal of repeating groups, and the primary key.

REVISIONREQUEST

request_id (PK)	grade_id (FK)	requested_points	reason	request_status	requested_at	reviewed_at	reviewed_by (FK)
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001
4	67	1	My answer matches the answer key	Closed		2026-10-11 15:30:00	P002

AUDITLOG

log_id (PK)	action	entity_type	entity_id	old_value	new_value	logged_at	details	actor_student_id	actor_teacher_id	admin_id
1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin			
2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001			
3	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to			
4	Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts	25-01894		

ADMIN

admin_id (PK)	admin_name	admin_email (AK)	admin_password_hash	admin_created_at
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	

GRADESHEET

student_id (PK)	assessment_id (PK)	grade_id (AK)	enrollment_id (AK)	student_name	student_email	student_password_hash	program	year_level	student_section	student_status
25-00418	1	1	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	2	2	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	3	3	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	4	121	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active

student_created_at	class_id	course_code	course_name	class_section	semester	academic_year	teacher_id	teacher_name	teacher_email	teacher_password_hash	department
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS

teacher_status	teacher_created_at	running_grade	enrolled_at	term	title	assessment_type	total_score	weight	assessment_created_at	score	remarks	grade_created_at	grade_updated_at
Active				Midterm	Quiz 1	Quiz	20	10		14			
Active				Midterm	Activity 1	Activity	50	10		41			
Active				Midterm	Midterm Exam	Exam	100	30		82			
Active				Finals	Quiz 2	Quiz	20	10					

4.3 Second Normal Form (2NF)

Identify partial dependencies and decompose tables as necessary. This is especially important when composite keys are present.

STUDENT

student_id (PK)	student_name	student_email (AK)	student_password_hash	program	year_level	student_section	student_status	student_created_at
25-00418	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-00731	Jerome Pagdanganan	jerome.pagdanganan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-01205	Ysabelle Macaraeg	ysabelle.macaraeg@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-01894	Lorenzo Villaflor	lorenzo.villaflor@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	

ASSESSMENT

assessment_id (PK)	term	title	assessment_type	total_score	weight	assessment_created_at	class_id	course_code
1	Midterm	Quiz 1	Quiz	20	10		1	CS 103
2	Midterm	Activity 1	Activity	50	10		1	CS 103
3	Midterm	Midterm Exam	Exam	100	30		1	CS 103
4	Finals	Quiz 2	Quiz	20	10		1	CS 103

course_name	class_section	semester	academic_year	teacher_id	teacher_name	teacher_email	teacher_password_hash	department	teacher_status
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active

teacher_created_at

ENROLLMENT

enrollment_id (PK)	student_id (AK, FK)	class_id (AK)	running_grade	enrolled_at
1	25-00418	1		
2	25-00731	1		
3	25-01205	1		
4	25-01894	1		

REVISIONREQUEST

request_id (PK)	grade_id (FK)	requested_points	reason	request_status	requested_at	reviewed_at	reviewed_by
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001

AUDITLOG

log_id (PK)	action	entity_type	entity_id	old_value	new_value	logged_at	details	actor_student_id	actor_teacher_id	admin_id
1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin			1
2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001			2
3	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to P002			2
4	Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts	25-01894		
5	Submitted revision request	RevisionRequest	2		Pending	2026-10-10 10:30:00	Activity 1: re-check	25-00418		

ENROLLMENT

enrollment_id (PK)	student_id (AK, FK)	class_id (AK, FK)	running_grade	enrolled_at
1	25-00418	1		
2	25-00731	1		
3	25-01205	1		
4	25-01894	1		
5	25-02156	1		

ADMIN

admin_id (PK)	admin_name	admin_email (AK)	admin_password_hash	admin_created_at
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
4	Geoff Andrei Mariano	geoff.mariano@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
5	Ron Jacob Puyat	ron.puyat@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	

STUDENTGRADE

grade_id (PK)	enrollment_id (AK, FK)	assessment_id (AK, FK)	score	remarks	grade_created_at	grade_updated_at
1	1	1	14			
2	1	2	41			
3	1	3	82			
4	2	1	19			

4.4 Third Normal Form (3NF)

Identify transitive dependencies and produce the final set of normalized relations.

STUDENT

student_id (PK)	full_name	email (AK)	password_hash	program	year_level	section	status	created_at
25-00418	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-00731	Jerome Pagdanganan	jerome.pagdanganan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-01205	Ysabelle Macaraeg	ysabelle.macaraeg@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	

TEACHER

teacher_id (PK)	full_name	email (AK)	password_hash	department	status	created_at
P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active	
P002	Rebecca R. Fajardo	rebecca.fajardo@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active	
P003	Benito Jose	benito.jose@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CAS	Active	

ADMIN

admin_id (PK)	full_name	email (AK)	password_hash	created_at
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
4	Geoff Andrei Mariano	geoff.mariano@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
5	Ron Jacob Puyat	ron.puyat@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	

COURSE

course_code (PK)	course_name
CS 103	Object-Oriented Programming (VB.NET with Database)
COMP 105	Information Management
CS 12	Software Engineering
GE 008	Ethics

CLASS

class_id (PK)	course_code (AK, FK)	section (AK)	semester (AK)	academic_year (AK)	teacher_id (FK)
1	CS 103	A	1st Semester	2026-2027	P001
2	CS 103	B	1st Semester	2026-2027	P001
3	COMP 105	A	1st Semester	2026-2027	P002
4	COMP 105	B	1st Semester	2026-2027	P002

ASSESSMENT

assessment_id (PK)	class_id (FK)	term	title	assessment_type	total_score	weight	created_at
1	1	Midterm	Quiz 1	Quiz	20	10	
2	1	Midterm	Activity 1	Activity	50	10	
3	1	Midterm	Midterm Exam	Exam	100	30	
4	1	Finals	Quiz 2	Quiz	20	10	
5	1	Finals	Project	Project	100	15	
6	1	Finals	Finals Exam	Exam	100	25	

STUDENTGRADE

grade_id (PK)	enrollment_id (AK, FK)	assessment_id (AK, FK)	score	remarks	created_at	updated_at
1	1	1	14			
2	1	2	41			
3	1	3	82			
4	2	1	19			
5	2	2	34			
6	2	3	93			

REVISIONREQUEST

request_id (PK)	grade_id (FK)	requested_points	reason	status	requested_at	reviewed_at	reviewed_by (FK)
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001
4	67	1	My answer matches the answer key	Closed		2026-10-11 15:30:00	P002

AUDITLOG

og_id (PK)	student_id (FK)	teacher_id (FK)	admin_id (FK)	action	entity_type	entity_id	old_value	new_value	logged_at	details
1			1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin
2			2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001
3			2	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to P002
4	25-01894			Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts
5	25-00418									

4.5 Functional Dependency Analysis

Table/Relation	Determinant	Dependent Attribute/s	Action / Normal Form Issue
Grade Sheet (UNORMALIZED)	student_id	classes, assessments, scores	Repeating groups and multi-valued cells (violates 1NF). Flattened to one row per student and assessment.
GRADE_SHEET_1NF	(student_id, assessment_id)	all other columns	Minimal candidate key. 1NF satisfied: atomic values, unique rows.
GRADE_SHEET_1NF	student_id	student name, email, password_hash, program, year_level, section, status, created_at	Partial dependency (2NF). Moved to Student.
GRADE_SHEET_1NF	assessment_id	class_id, term, title, assessment_type, total_score, weight, created_at, and the class, course and teacher details	Partial dependency (2NF). Moved to Assessment.

GRADE_SHEET_1NF	enrollment_id	student_id, class_id, running_grade, enrolled_at	Partial dependency (2NF). Moved to Enrollment.
GRADE_SHEET_1NF	(enrollment_id, assessment_id)	grade_id, score, remarks, created_at, updated_at	Full dependency. Kept as StudentGrade.
Assessment (2NF)	class_id	course_code, section, semester, academic_year, teacher_id	Transitive dependency (assessment_id → class_id → class details). Moved to Class (3NF).
Class	course_code	course_name	Transitive dependency (class_id → course_code → course_name). Moved to Course (3NF).
Class	teacher_id	teacher name, email, password_hash, department, status, created_at	Transitive dependency (class_id → teacher_id → teacher details). Moved to Teacher (3NF).
Final 10 tables (see 4.6)	Each table's primary key	All other attributes of that table	Already in 3NF: no partial or transitive dependency. running_grade in Enrollment is derived, so the backend must keep it updated.

4.6 Final 3NF Tables

Table Name	Primary Key	Foreign Key/s	Major Non-Key Attributes	Purpose
Student	student_id	None	full_name, email, password_hash, program, year_level, section, status, created_at	Stores student accounts. Students log in to view their grades and send revision requests.
Teacher	teacher_id	None	full_name, email, password_hash, department, status, created_at	Stores teacher accounts. Teachers handle classes, record scores and review revision requests.
Admin	admin_id	None	full_name, email,	Stores admin

			password_hash, created_at	accounts. Admins manage student and teacher accounts and class assignments.
Course	course_code	None	course_name	Stores the subjects offered, such as CS 103 Object-Oriented Programming.
Class	class_id	course_code (Course), teacher_id (Teacher)	section, semester, academic_year	One offering of a course for a section, semester and academic year, handled by one teacher.
Enrollment	enrollment_id	student_id (Student), class_id (Class)	running_grade, enrolled_at	Places a student in a class. Resolves the many-to-many between Student and Class. A student can enroll in a class only once.
Assessment	assessment_id	class_id (Class)	term, title, assessment_type, total_score, weight, created_at	A gradable item in a class (quiz, activity, exam, project) with its term and weight.
StudentGrade	grade_id	enrollment_id (Enrollment), assessment_id (Assessment)	score, remarks, created_at, updated_at	One student's score on one assessment. Resolves the many-to-many between Enrollment and Assessment. One score per student per assessment.
RevisionRequest	request_id	grade_id (StudentGrade), reviewed_by	requested_points, reason, status, requested_at,	A student's request to re-check a score, with its review

		(Teacher)	reviewed_at	status. reviewed_by stays empty until a teacher reviews it.
AuditLog	log_id	student_id (Student), teacher_id (Teacher), admin_id (Admin)	action, entity_type, entity_id, old_value, new_value, logged_at, details	Permanent record of who did what. Exactly one actor ID is filled per entry.

5. Data Dictionary

Document every field in every final 3NF table. The data dictionary must match the actual MySQL implementation.

Table	Field	Data Type	Size	PK/FK	Null?	Default	Constraint	Description
Student	student_id	VARCHAR	10	PK	NO	-	PRIMARY KEY	Student ID number (e.g. 25-06767)
Student	full_name	VARCHAR	100	-	NO	-	-	Student's full name
Student	email	VARCHAR	100	-	NO	-	UNIQUE	School email address
Student	password_hash	VARCHAR	255	-	NO	-	-	Hashed login password
Student	program	VARCHAR	100	-	NO	-	-	Degree program (e.g. BS Computer Science)
Student	year_level	TINYINT UNSIGNED	-	-	NO	-	-	Year level of the student
Student	section	VARCHAR	20	-	NO	-	-	Section (e.g. BSCS

								2A)
Student	status	ENUM	-	-	NO	‘Active’	Values: Active, Inactive	Whether the account is in use
Student	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
Teacher	teacher_id	VARCHAR	10	PK	NO	-	PRIMARY KEY	Teacher ID (e.g. P001)
Teacher	full_name	VARCHAR	100	-	NO	-	-	Teacher's full name
Teacher	email	VARCHAR	100	-	NO	-	UNIQUE	School email address
Teacher	password_hash	VARCHAR	255	-	NO	-	-	Hashed login password
Teacher	department	VARCHAR	50	-	NO	-	-	Department the teacher belongs to
Teacher	status	ENUM	-	-	NO	‘Active’	Values: Active, Inactive	Whether the account is in use
Teacher	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
Admin	admin_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREME	Admin identifier

							NT	
Admin	full_name	VARCHAR	100	-	NO	-	-	Admin's full name
Admin	email	VARCHAR	100	-	NO	-	UNIQUE	Admin email address
Admin	password_hash	VARCHAR	255	-	NO	-	-	Hashed login password
Admin	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
Course	course_code	VARCHAR	20	PK	NO	-	PRIMARY KEY	Course code (e.g. CS 103)
Course	course_name	VARCHAR	100	-	NO	-	-	Course title (e.g. Object Oriented Programming)
Class	class_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Class identifier
Class	course_code	VARCHAR	20	FK	NO	-	FOREIGN KEY →Course(course_code), ON UPDATE CASCADE, ON DELETE RESTRICT;part	Course this class belongs to

							of UNIQUE uq_class	
Class	section	VARCHAR	20	-	NO	-	Part of UNIQUE uq_class	Section (e.g. BSCS 2A)
Class	semester	ENUM	-	-	NO	-	Values:1st Semester, 2nd Semester; part of UNIQUE uq_class	Semester the class runs in
Class	academic_year	VARCHAR	9	-	NO	-	Part of UNIQUE uq_class	Academic year (e.g. 2026-2027)
Class	teacher_id	VARCHAR	10	FK	NO	-	FOREIGN KEY →Teacher (teacher_id),ON UPDATE CASCADE, ON DELETE RESTRICT	Teacher handling the class
Enrollment	enrollment_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Enrollment identifier
Enrollment	student_id	VARCHAR	10	FK	NO	-	FOREIGN KEY →Student (student_i	Enrolled student

							d), ON UPDATE CASCADE, ON DELETE RESTRICT; part of UNIQUEuq_enrollment	
Enrollment	class_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY →Class (class_id), ON UPDATE CASCADE, ON DELETE RESTRICT; part of UNIQUEuq_enrollment	Class the student is enrolled in
Enrollment	running_grade	DECIMAL	5,2	-	YES	NULL	-	Overall standing in the class; NULL until the first score is recorded
Enrollment	enrolled_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the student was enrolled

Assessment	assessment_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Assessment identifier
Assessment	class_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → Class(class_id), ON UPDATE CASCADE, ON DELETE CASCADE	Class the assessment belongs to
Assessment	term	ENUM	-	-	NO	-	Values: Midterm, Finals	Grading period
Assessment	title	VARCHAR	100	-	NO	-	-	Assessment name (e.g. Quiz 2)
Assessment	assessment_type	ENUM	-	-	NO	-	Values: Quiz, Activity, Exam, Project, Attendance	Kind of assessment
Assessment	total_score	DECIMAL	6,2	-	NO	-	CHECK (total_score > 0)	Highest possible score
Assessment	weight	DECIMAL	5,2	-	NO	-	CHECK (weight BETWEE	Percentage weight (e.g.

							N 0 AND 100)	15.00)
Assessment	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
StudentGrade	grade_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Grade identifier
StudentGrade	enrollment_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → Enrollment(enrollment_id), ON UPDATECASCADE, ON DELETECASCADE; part of UNIQUE uq_grade	Enrollment the grade belongs to
StudentGrade	assessment_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → Assessment(assessment_id), ON UPDATECASCADE, ON DELETE	Assessment being graded

							CASCADE; part of UNIQUE uq_grade	
StudentGrade	score	DECIMAL	6,2	-	YES	NULL	-	Score earned; NULL until graded
StudentGrade	remarks	VARCHAR	255	-	YES	NULL	-	Optional teacher remarks
StudentGrade	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the grade row was created
StudentGrade	updated_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	When the grade row was last changed
RevisionRequest	request_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Request identifier
RevisionRequest	grade_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → StudentGrade(grad_id), ON UPDATE CASCADE, ON	Grade being questioned

							DELETE CASCADE	
RevisionRequest	requested_points	DECIMAL	5,2	-	YES	NULL	-	Points asked for (e.g. 2.00); NULL for a plain re-check
RevisionRequest	reason	VARCHAR	255	-	NO	-	-	Student's reason for the request
RevisionRequest	status	ENUM	-	-	NO	'Pending'	Values: Pending, Under Review, Approved ,Closed	Where the request stands
RevisionRequest	requested_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the request was filled
RevisionRequest	reviewed_at	TIMESTAMP	-	-	YES	NULL	-	When the teacher reviewed it
RevisionRequest	reviewed_by	VARCHAR	10	FK	YES	NULL	FOREIGN KEY → Teacher(teacher_id), ON UPDATE CASCADE, ON DELETE SET NULL	Teacher who reviewed it
AuditLog	log_id	BEGINT UNSIGNED	-	PK	NO	-	PRIMARY KEY,	Log entry identifier

							AUTO_INCREMENT	
AuditLog	student_id	VARCHAR	10	FK	YES	NULL	FOREIGN KEY → Student(student_id), ON DELETE RESTRICT; CHECK exactly one actor	Student who did the action
AuditLog	teacher_id	VARCHAR	10	FK	YES	NULL	FOREIGN KEY → Teacher(teacher_id), ON DELETE RESTRICT; CHECK exactly one actor	Teacher who did the action
AuditLog	admin_id	INT UNSIGNED	-	FK	YES	NULL	FOREIGN KEY → Admin(admin_id), ON DELETE RESTRICT; CHECK exactly one actor	Admin who did the action
AuditLog	action	VARCHAR	100	-	NO	-	-	What was

								done (e.g. Grade updated)
AuditLog	entity_type	VARCHAR	50	-	NO	-	INDEX idx_entity	Kind of record affected (not a foreign key)
AuditLog	entity_id	VARCHAR	20	-	NO	-	INDEX idx_entity	ID of the record affected (not a foreign key)
AuditLog	old_value	VARCHAR	100	-	YES	NULL	-	Value before a score change (e.g. 16/20)
AuditLog	new_value	VARCHAR	100	-	YES	NULL	-	Value after a score change (e.g. 18/20)
AuditLog	logged_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	What the action happened
AuditLog	details	VARCHAR	500	-	YES	NULL	-	Extra notes about the action

6. MySQL Database Implementation

Implement the approved 3NF design in MySQL/MariaDB. The implementation must be consistent with the ERD and data dictionary.

6.1 Database Creation

Provide the CREATE DATABASE statement and database name.

```
MariaDB [(none)]> create database centralized_and_transparent_grading_system
    -> character set utf8mb4
    -> collate utf8mb4_unicode_ci;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]>
```

6.2 DDL – Table Creation

Provide CREATE TABLE statements showing data types, primary keys, foreign keys, NOT NULL, UNIQUE, DEFAULT, CHECK, AUTO_INCREMENT, and other appropriate constraints.

```
MariaDB [centralized_and_transparent_grading_system]> create table Student(
    -> student_id varchar(10) primary key,
    -> full_name varchar(100) not null,
    -> email varchar(100) not null unique,
    -> password_hash varchar(255) not null,
    -> program varchar(100) not null,
    -> year_level tinyint unsigned not null,
    -> section varchar(20) not null,
    -> status enum('Active','Inactive
    '> ') not null default 'Active',
    -> created_at timestamp not null default CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.076 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> alter table Student
    -> modify column status ENUM('Active', 'Inactive')
    -> not null default 'Active';
Query OK, 21 rows affected (0.081 sec)
Records: 21  Duplicates: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Teacher(
    -> teacher_id varchar(10) primary key,
    -> full_name varchar(100) not null,
    -> email varchar(100) not null unique,
    -> password_hash varchar(255) not null,
    -> department varchar(50) not null,
    -> status enum('Active','Inactive') not null default 'Active',
    -> created_at timestamp not null default CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.064 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Admin(
    -> admin_id int unsigned primary key auto_increment,
    -> full_name varchar(100) not null,
    -> email varchar(100) not null unique,
    -> password_hash varchar(255) not null,
    -> created_at timestamp not null default CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.037 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Course(
    -> course_code varchar(20) primary key,
    -> course_name varchar(100) not null);
Query OK, 0 rows affected (0.012 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Class(
    -> class_id int unsigned primary key auto_increment,
    -> course_code varchar(20) not null,
    -> section varchar(20) not null,
    -> semester enum('1st Semester', '2nd Semester') not null,
    -> academic_year varchar(9) not null,
    -> teacher_id varchar(10) not null,
    -> constraint fk_class_course
    -> Foreign Key(course_code) references Course(course_code)
    -> on update cascade on delete restrict,
    -> constraint fk_class_teacher
    -> Foreign Key(teacher_id) references Teacher(teacher_id)
    -> on update cascade on delete restrict,
    -> unique key uq_class (course_code, section, semester, academic_year));
Query OK, 0 rows affected (0.083 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Enrollment(
    -> enrollment_id int unsigned primary key auto_increment,
    -> student_id varchar(10) not null,
    -> class_id int unsigned not null,
    -> running_grade decimal(5,2) null,
    -> enrolled_at timestamp not null default CURRENT_TIMESTAMP,
    -> constraint fk_enrollment_student
    -> foreign key(student_id) references Student(student_id)
    -> on update cascade on delete restrict,
    -> constraint fk_enrollment_class
    -> foreign key(class_id) references Class(class_id)
    -> on update cascade on delete restrict,
    -> unique key uq_enrollment(student_id, class_id));
Query OK, 0 rows affected (0.075 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Assessment(
    -> assessment_id int unsigned primary key auto_increment,
    -> class_id int unsigned not null,
    -> term enum('Midterm','Finals') not null,
    -> title varchar(100) not null,
    -> assessment_type enum('Quiz','Activity','Exam','Project','Attendance') not null,
    -> total_score decimal(6,2) not null,
    -> weight decimal(5,2) not null,
    -> created_at timestamp not null default CURRENT_TIMESTAMP,
    -> constraint fk_assessment_class
    -> foreign key(class_id) references Class(class_id)
    -> on update cascade on delete cascade,
    -> constraint chk_weight check(weight BETWEEN 0 AND 100),
    -> constraint chk_total_score check(total_score > 0));
Query OK, 0 rows affected (0.070 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table StudentGrade(
    -> grade_id int unsigned primary key auto_increment,
    -> enrollment_id int unsigned not null,
    -> assessment_id int unsigned not null,
    -> score decimal(6,2) null,
    -> remarks varchar(255) null,
    -> created_at timestamp not null default CURRENT_TIMESTAMP,
    -> updated_at timestamp not null default CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    -> constraint fk_grade_enrollment
    -> foreign key(enrollment_id) references Enrollment(enrollment_id)
    -> on update cascade on delete cascade,
    -> constraint fk_grade_assessment
    -> foreign key(assessment_id) references Assessment(assessment_id)
    -> on update cascade on delete cascade,
    -> unique key uq_grade(enrollment_id, assessment_id));
Query OK, 0 rows affected (0.036 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table RevisionRequest(
    -> request_id int unsigned primary key auto_increment,
    -> grade_id int unsigned not null,
    -> requested_points decimal(5,2) null,
    -> reason varchar(255) not null,
    -> status enum('Pending','Under Review','Approved','Closed') not null default 'Pending',
    -> requested_at timestamp not null default CURRENT_TIMESTAMP,
    -> reviewed_at timestamp null,
    -> reviewed_by varchar(10) null,
    -> constraint fk_request_grade
    -> foreign key(grade_id) references StudentGrade(grade_id)
    -> on update cascade on delete cascade,
    -> constraint fk_request_teacher
    -> foreign key(reviewed_by) references Teacher(teacher_id)
    -> on update cascade on delete set null);
Query OK, 0 rows affected (0.082 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table AuditLog(
    -> log_id bigint unsigned primary key auto_increment,
    -> student_id varchar(10) null,
    -> teacher_id varchar(10) null,
    -> admin_id int unsigned null,
    -> action varchar(100) not null,
    -> entity_type varchar(50) not null,
    -> entity_id varchar(20) not null,
    -> old_value varchar(100) null,
    -> new_value varchar(100) null,
    -> logged_at timestamp not null default CURRENT_TIMESTAMP,
    -> details varchar(500) null,
    -> constraint fk_audit_student
    -> foreign key(student_id) references Student(student_id)
    -> on delete restrict,
    -> constraint fk_audit_teacher
    -> foreign key(teacher_id) references Teacher(teacher_id)
    -> on delete restrict,
    -> constraint fk_audit_admin
    -> foreign key(admin_id) references Admin(admin_id)
    -> on delete restrict,
    -> constraint chk_audit_one_actor
    -> check((student_id is not null) + (teacher_id is not null) + (admin_id is not null) = 1),
    -> index idx_entity (entity_type, entity_id));
Query OK, 0 rows affected (0.069 sec)
```

6.3 DML – Sample Data

Provide INSERT statements or approved import/bulk-load method. Include enough records to demonstrate all major system functions.

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Admin.csv'
    -> INTO TABLE Admin
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (full_name, email, password_hash);
Query OK, 5 rows affected (0.039 sec)
Records: 5  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Teacher.csv'
    -> INTO TABLE Teacher
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (teacher_id, full_name, email, password_hash, department);
Query OK, 3 rows affected (0.044 sec)
Records: 3  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Student.csv'
    -> INTO TABLE Student
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (student_id, full_name, email, password_hash, program, year_level, section);
Query OK, 21 rows affected (0.041 sec)
Records: 21  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Course.csv'
    -> INTO TABLE Course
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (course_code, course_name);
Query OK, 4 rows affected (0.054 sec)
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Class.csv'
    -> INTO TABLE Class
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (course_code, section, semester, academic_year, teacher_id);
Query OK, 4 rows affected (0.052 sec)
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Enrollment.csv'
    -> INTO TABLE Enrollment
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (student_id, class_id);
Query OK, 40 rows affected (0.044 sec)
Records: 40  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Assessment.csv'
    -> INTO TABLE Assessment
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (class_id, term, title, assessment_type, total_score, weight);
Query OK, 24 rows affected (0.045 sec)
Records: 24  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/StudentGrade.csv'
    -> INTO TABLE StudentGrade
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (enrollment_id, assessment_id, score);
Query OK, 120 rows affected (0.050 sec)
Records: 120  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> insert into StudentGrade(enrollment_id, assessment_id, score)
    -> select
    -> e.enrollment_id,
    -> a.assessment_id,
    -> null
    -> FROM Enrollment e
    -> JOIN Assessment a
    -> ON a.class_id = e.class_id
    -> WHERE a.term = 'Finals'
    -> AND NOT EXISTS(
    -> SELECT 1
    -> FROM StudentGrade sg
    -> WHERE sg.enrollment_id = e.enrollment_id
    -> AND sg.assessment_id = a.assessment_id);
Query OK, 120 rows affected (0.047 sec)
Records: 120  Duplicates: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/RevisionRequest.cs
v'
    -> INTO TABLE RevisionRequest
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (grade_id, requested_points, reason, status, reviewed_at, reviewed_by);
Query OK, 4 rows affected (0.043 sec)
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/AuditLog.csv'
    -> INTO TABLE AuditLog
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (student_id, teacher_id, admin_id, action, entity_type, entity_id, old_value, new_value, logged_at, details);
Query OK, 11 rows affected (0.048 sec)
Records: 11  Deleted: 0  Skipped: 0  Warnings: 0
```

6.4 Required Queries

Provide meaningful SELECT queries using filtering, sorting, joins, aggregate functions, GROUP BY/HAVING, and other queries appropriate to the project.

```
MariaDB [centralized_and_transparent_grading_system]> select student_id, full_name, email
    -> from student
    -> where program = 'Computer Science'
    -> AND year_level = 2
    -> AND section = 'A';
ERROR 2006 (HY000): MySQL server has gone away
No connection. Trying to reconnect...
Connection id:    37
Current database: centralized_and_transparent_grading_system

+------------+--------------------------+-------------------------------+
| student_id | full_name                | email                         |
+------------+--------------------------+-------------------------------+
| 25-00418   | Kyla Mae Dimaculangan    | kyla.dimaculangan@plp.com     |
| 25-00731   | Jerome Pagdanganan       | jerome.pagdanganan@plp.com    |
| 25-01205   | Ysabelle Macaraeg        | ysabelle.macaraeg@plp.com     |
| 25-01894   | Lorenzo Villaflor        | lorenzo.villaflor@plp.com     |
| 25-02156   | Trisha Anne Alcantara    | trisha.alcantara@plp.com      |
| 25-02643   | Nathaniel Evangelista    | nathaniel.evangelista@plp.com |
| 25-03087   | Princess Joy Magbanua    | princess.magbanua@plp.com     |
| 25-03519   | Miguel Andre Panganiban  | miguel.panganiban@plp.com     |
| 25-04072   | Samantha Rose Sarmiento  | samantha.sarmiento@plp.com    |
| 25-04396   | Gabriel Zandro Natividad | gabriel.natividad@plp.com     |
| 25-05128   | Jasmine Faye Catapang    | jasmine.catapang@plp.com      |
| 25-05764   | Elijah Rebolledo         | elijah.rebolledo@plp.com      |
| 25-06291   | Daniella Abellera        | daniella.abellera@plp.com     |
+------------+--------------------------+-------------------------------+
13 rows in set (0.082 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> select class_id, COUNT(*) AS enrolled_students
    -> FROM Enrollment
    -> GROUP BY class_id
    -> ORDER BY class_id;
+----------+------------------+
| class_id | enrolled_students |
+----------+------------------+
|        1 |               12 |
|        2 |                8 |
|        3 |               12 |
|        4 |                8 |
+----------+------------------+
4 rows in set (0.024 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> select class_id, COUNT(*) AS enrolled_students
    -> FROM Enrollment
    -> GROUP BY class_id
    -> HAVING COUNT(*) > 10
    -> ORDER BY enrolled_students DESC;
+----------+------------------+
| class_id | enrolled_students |
+----------+------------------+
|        1 |               12 |
|        3 |               12 |
+----------+------------------+
2 rows in set (0.012 sec)
```

6.5 Database Evidence

Insert screenshots of the schema/tables, sample records, relationships/constraints, and selected query results.

```
MariaDB [centralized_and_transparent_grading_system]> show tables;
+-----------------------------------------------------------+
| Tables_in_centralized_and_transparent_grading_system |
+-----------------------------------------------------------+
| admin                                                     |
| assessment                                                |
| auditlog                                                  |
| class                                                     |
| course                                                    |
| enrollment                                                |
| revisionrequest                                           |
| student                                                   |
| studentgrade                                              |
| teacher                                                   |
+-----------------------------------------------------------+
10 rows in set (0.031 sec)

MariaDB [centralized_and_transparent_grading_system]>
```

```
MariaDB [centralized_and_transparent_grading_system]> select * from student
    -> where section = 'A'
    -> limit 7;
+------------+---------------------+-----------------------------+------------------------------+------------------+------------+---------+--------+---------------------+
| student_id | full_name           | email                       | password_hash                | program          | year_level | section | status | created_at          |
+------------+---------------------+-----------------------------+------------------------------+------------------+------------+---------+--------+---------------------+
| 25-00418   | Kyla Mae Dimaculangan | kyla.dimaculangan@plp.com   | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-00731   | Jerome Pagdanganan  | jerome.pagdanganan@plp.com  | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-01205   | Ysabelle Macaraeg   | ysabelle.macaraeg@plp.com   | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-01894   | Lorenzo Villaflor   | lorenzo.villaflor@plp.com   | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-02156   | Trisha Anne Alcantara | trisha.alcantara@plp.com  | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-02643   | Nathaniel Evangelista | nathaniel.evangelista@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-03087   | Princess Joy Magbanua | princess.magbanua@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
+------------+---------------------+-----------------------------+------------------------------+------------------+------------+---------+--------+---------------------+
7 rows in set (0.012 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> select * from teacher;
+------------+--------------------+-----------------------------------+------------------------------+------------+--------+---------------------+
| teacher_id | full_name          | email                             | password_hash                | department | status | created_at          |
+------------+--------------------+-----------------------------------+------------------------------+------------+--------+---------------------+
| P001       | Jhun Alvarez       | jhun.alvarez@dauntlesswolves.test | REPLACE_WITH_HASH_FROM_BACKEND | CCS        | Active | 2026-10-14 21:44:38 |
| P002       | Rebecca R. Fajardo | rebecca.fajardo@dauntlesswolves.test | REPLACE_WITH_HASH_FROM_BACKEND | CCS        | Active | 2026-10-14 21:44:38 |
| P003       | Benito Jose        | benito.jose@dauntlesswolves.test  | REPLACE_WITH_HASH_FROM_BACKEND | CAS        | Active | 2026-10-14 21:44:38 |
+------------+--------------------+-----------------------------------+------------------------------+------------+--------+---------------------+
3 rows in set (0.008 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> SELECT * FROM vw_class_grade_sheet
    -> WHERE class_id = 1 AND term = 'Midterm'
    -> ORDER BY full_name, assessment_id;
+----------+------------+-------------------------+---------------+---------+--------------+-------+-------------+--------+
| class_id | student_id | full_name               | assessment_id | term    | title        | score | total_score | weight |
+----------+------------+-------------------------+---------------+---------+--------------+-------+-------------+--------+
|        1 | 25-05764   | Elijah Rebolledo        |             1 | Midterm | Quiz 1       | 16.00 |       20.00 |  10.00 |
|        1 | 25-05764   | Elijah Rebolledo        |             2 | Midterm | Activity 1   | 34.00 |       50.00 |  10.00 |
|        1 | 25-05764   | Elijah Rebolledo        |             3 | Midterm | Midterm Exam | 76.00 |      100.00 |  30.00 |
|        1 | 25-04396   | Gabriel Zandro Natividad |             1 | Midterm | Quiz 1       | 14.00 |       20.00 |  10.00 |
|        1 | 25-04396   | Gabriel Zandro Natividad |             2 | Midterm | Activity 1   | 37.00 |       50.00 |  10.00 |
|        1 | 25-04396   | Gabriel Zandro Natividad |             3 | Midterm | Midterm Exam | 97.00 |      100.00 |  30.00 |
|        1 | 25-05128   | Jasmine Faye Catapang   |             1 | Midterm | Quiz 1       | 19.00 |       20.00 |  10.00 |
|        1 | 25-05128   | Jasmine Faye Catapang   |             2 | Midterm | Activity 1   | 46.00 |       50.00 |  10.00 |
|        1 | 25-05128   | Jasmine Faye Catapang   |             3 | Midterm | Midterm Exam | 98.00 |      100.00 |  30.00 |
|        1 | 25-00731   | Jerome Pagdanganan      |             1 | Midterm | Quiz 1       | 19.00 |       20.00 |  10.00 |
|        1 | 25-00731   | Jerome Pagdanganan      |             2 | Midterm | Activity 1   | 34.00 |       50.00 |  10.00 |
|        1 | 25-00731   | Jerome Pagdanganan      |             3 | Midterm | Midterm Exam | 93.00 |      100.00 |  30.00 |
|        1 | 25-00418   | Kyla Mae Dimaculangan   |             1 | Midterm | Quiz 1       | 14.00 |       20.00 |  10.00 |
|        1 | 25-00418   | Kyla Mae Dimaculangan   |             2 | Midterm | Activity 1   | 41.00 |       50.00 |  10.00 |
|        1 | 25-00418   | Kyla Mae Dimaculangan   |             3 | Midterm | Midterm Exam | 82.00 |      100.00 |  30.00 |
|        1 | 25-01894   | Lorenzo Villaflor       |             1 | Midterm | Quiz 1       |  9.00 |       20.00 |  10.00 |
|        1 | 25-01894   | Lorenzo Villaflor       |             2 | Midterm | Activity 1   | 23.00 |       50.00 |  10.00 |
|        1 | 25-01894   | Lorenzo Villaflor       |             3 | Midterm | Midterm Exam | 45.00 |      100.00 |  30.00 |
|        1 | 25-03519   | Miguel Andre Panganiban |             1 | Midterm | Quiz 1       | 14.00 |       20.00 |  10.00 |
|        1 | 25-03519   | Miguel Andre Panganiban |             2 | Midterm | Activity 1   | 46.00 |       50.00 |  10.00 |
|        1 | 25-03519   | Miguel Andre Panganiban |             3 | Midterm | Midterm Exam | 80.00 |      100.00 |  30.00 |
|        1 | 25-02643   | Nathaniel Evangelista   |             1 | Midterm | Quiz 1       | 18.00 |       20.00 |  10.00 |
|        1 | 25-02643   | Nathaniel Evangelista   |             2 | Midterm | Activity 1   | 45.00 |       50.00 |  10.00 |
|        1 | 25-02643   | Nathaniel Evangelista   |             3 | Midterm | Midterm Exam | 90.00 |      100.00 |  30.00 |
|        1 | 25-03087   | Princess Joy Magbanua   |             1 | Midterm | Quiz 1       | 17.00 |       20.00 |  10.00 |
|        1 | 25-03087   | Princess Joy Magbanua   |             2 | Midterm | Activity 1   | 36.00 |       50.00 |  10.00 |
|        1 | 25-03087   | Princess Joy Magbanua   |             3 | Midterm | Midterm Exam | 85.00 |      100.00 |  30.00 |
|        1 | 25-04072   | Samantha Rose Sarmiento |             1 | Midterm | Quiz 1       | 18.00 |       20.00 |  10.00 |
|        1 | 25-04072   | Samantha Rose Sarmiento |             2 | Midterm | Activity 1   | 44.00 |       50.00 |  10.00 |
|        1 | 25-04072   | Samantha Rose Sarmiento |             3 | Midterm | Midterm Exam | 87.00 |      100.00 |  30.00 |
|        1 | 25-02156   | Trisha Anne Alcantara   |             1 | Midterm | Quiz 1       | 17.00 |       20.00 |  10.00 |
|        1 | 25-02156   | Trisha Anne Alcantara   |             2 | Midterm | Activity 1   | 45.00 |       50.00 |  10.00 |
|        1 | 25-02156   | Trisha Anne Alcantara   |             3 | Midterm | Midterm Exam | 83.00 |      100.00 |  30.00 |
|        1 | 25-01205   | Ysabelle Macaraeg       |             1 | Midterm | Quiz 1       | 17.00 |       20.00 |  10.00 |
|        1 | 25-01205   | Ysabelle Macaraeg       |             2 | Midterm | Activity 1   | 36.00 |       50.00 |  10.00 |
|        1 | 25-01205   | Ysabelle Macaraeg       |             3 | Midterm | Midterm Exam | 85.00 |      100.00 |  30.00 |
+----------+------------+-------------------------+---------------+---------+--------------+-------+-------------+--------+
36 rows in set (0.009 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> SELECT * FROM vw_open_revision_requests ORDER BY requested_at;
+------------+-------------------+--------------+-------+-----------------+-----------------------------+--------------+---------------------+
| request_id | full_name         | title        | score | requested_points | reason                      | status       | requested_at        |
+------------+-------------------+--------------+-------+-----------------+-----------------------------+--------------+---------------------+
|          1 | Lorenzo Villaflor | Quiz 1       |  9.00 |            2.00 | Misread answer on item 5    | Pending      | 2026-10-14 22:38:53 |
|          2 | Kyla Mae Dimaculangan | Activity 1 | 41.00 |            NULL | Please re-check my total score | Under Review | 2026-10-14 22:38:53 |
+------------+-------------------+--------------+-------+-----------------+-----------------------------+--------------+---------------------+
2 rows in set (0.001 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> SELECT * FROM vw_grade_history ORDER BY logged_at DESC;
+--------+----------+-------------------------+----------------+-----------+-----------+--------------+---------------------+
| log_id | actor_id | action                  | entity_type    | entity_id | old_value | new_value    | logged_at           |
+--------+----------+-------------------------+----------------+-----------+-----------+--------------+---------------------+
|      9 | P001     | Updated request status  | RevisionRequest | 3         | Pending   | Approved     | 2026-10-11 15:30:00 |
|     10 | P002     | Updated request status  | RevisionRequest | 4         | Pending   | Closed       | 2026-10-11 15:30:00 |
|     11 | P001     | Updated score           | StudentGrade   | 6         | 88/100    | 93/100       | 2026-10-11 15:30:00 |
|      8 | P001     | Updated request status  | RevisionRequest | 2         | Pending   | Under Review | 2026-10-11 08:50:00 |
+--------+----------+-------------------------+----------------+-----------+-----------+--------------+---------------------+
4 rows in set (0.002 sec)

MariaDB [centralized_and_transparent_grading_system]>
```

7. VB.NET System Implementation

Develop a working VB.NET front-end connected to the MySQL database. The interface must support the approved project scope and enforce relevant business rules.

7.1 Reports / Outputs

Document reports, summaries, printable forms, dashboards, or exports included in the system.

7.1 Reports / Outputs

Professor

- **Class Grade Sheet:** displays students' assessment scores, running grades, and academic status. It can be filtered and **exported as a PDF**.
- **Class Performance Summary (Analytics):** displays the class average, number of passing students, number of at-risk students, and highest grade.
- **Averages by Assessment Type:** displays average scores for quizzes, activities, examinations, and projects using progress bars.
- **Audit Log – Recent Changes:** records score updates and request status changes, including the affected student, item, date and time, and user responsible. It can be **exported as a PDF**.
- **Digital Gradebook:** displays students' raw scores, percentage scores, assessment results, and running grades. It also **supports PDF export**. Replicating the original Professor grading sheet in Excel.
- **Student Grade Record:** displays a selected student's assessment scores and allows professors to record scores and recalculate grades.
- **Clarification Requests:** displays students' grade clarification requests, including the assessment, current score, requested adjustment, reason, and request status.

Student

- **My Grades (Grade Report):** displays assessment scores, relative weights, running grades, and academic status. Students can filter grades by midterm, finals, or combined results and select a course.
- **Grade History:** displays recorded grade changes, including the date, assessment, old score, new score, person responsible, and reason.
- **My Performance Overview:** displays the current average, activities passed, open clarification requests, and highest score. Progress bars show performance in quizzes, activities, examinations, and projects.
- **Clarification Request Status:** displays submitted requests, including the course, assessment, requested action, and current status. It also shows the reviewer's response.
- **Student Profile:** displays the student's account information, including student ID, full name, email, program, year level, section, and account status.
- **Change Password:** allows the student to change their password

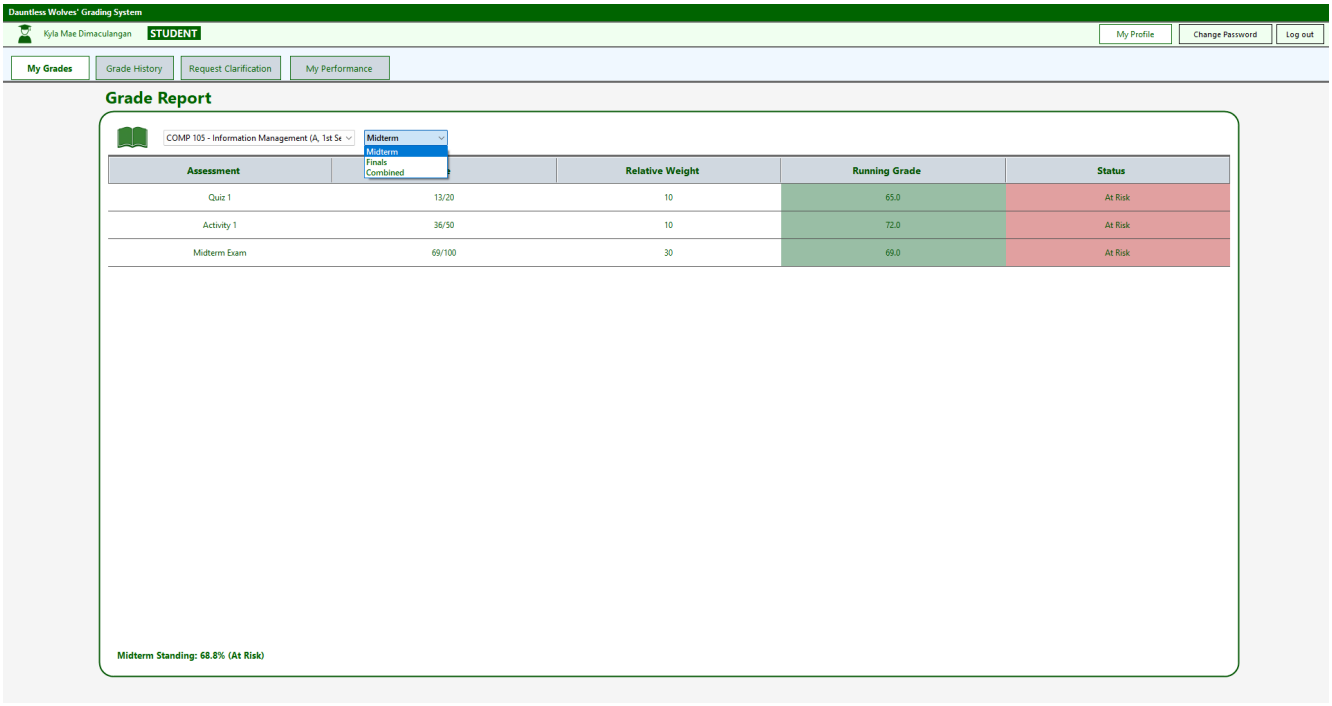
Admin

- **Admin Dashboard:** displays the total number of students, professors, active classes, and pending clarification requests. It also shows recent system updates and provides quick access to administrative functions.
- **Student Records:** displays student IDs, full names, year levels, sections, programs, and account statuses. Administrators can add, edit, search, filter, and delete student records.
- **Professor Records:** displays professor names, employee IDs, departments, assigned classes, employment types, and account statuses. Administrators can add, edit, search, filter, and delete professor records.
- **Class/Subject Management:** allows administrators to create classes by entering the subject code, subject name, section, assigned professor, semester, and academic year.
- **Grade Entry:** allows administrators to select a student, subject, and assessment before entering and saving grades.
- **Clarification Requests:** displays students' grade clarification requests, including the assessment, current score, requested adjustment, reason, and request status. Administrators can review requests and view the professor's response.
- **Audit Logs:** records administrative and system activities, including the user responsible, action performed, affected entity, previous and updated values, date, time, and additional details. Records can be filtered and exported as CSV or PDF.

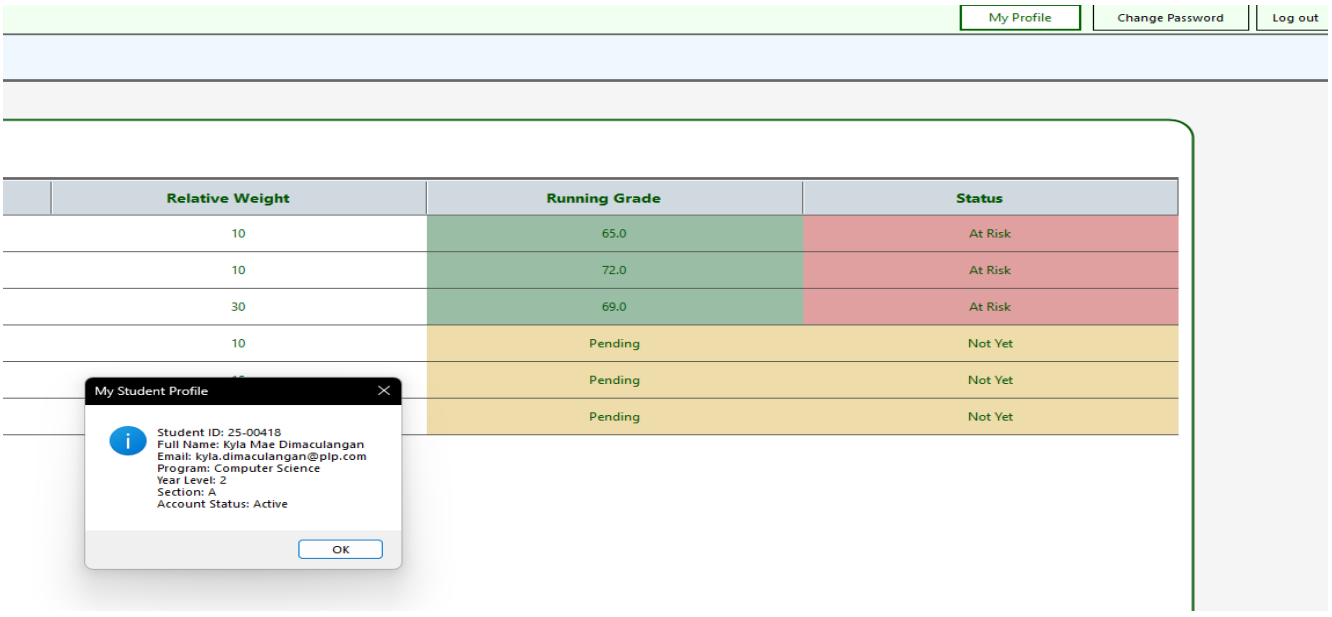
7.2 System Screenshots

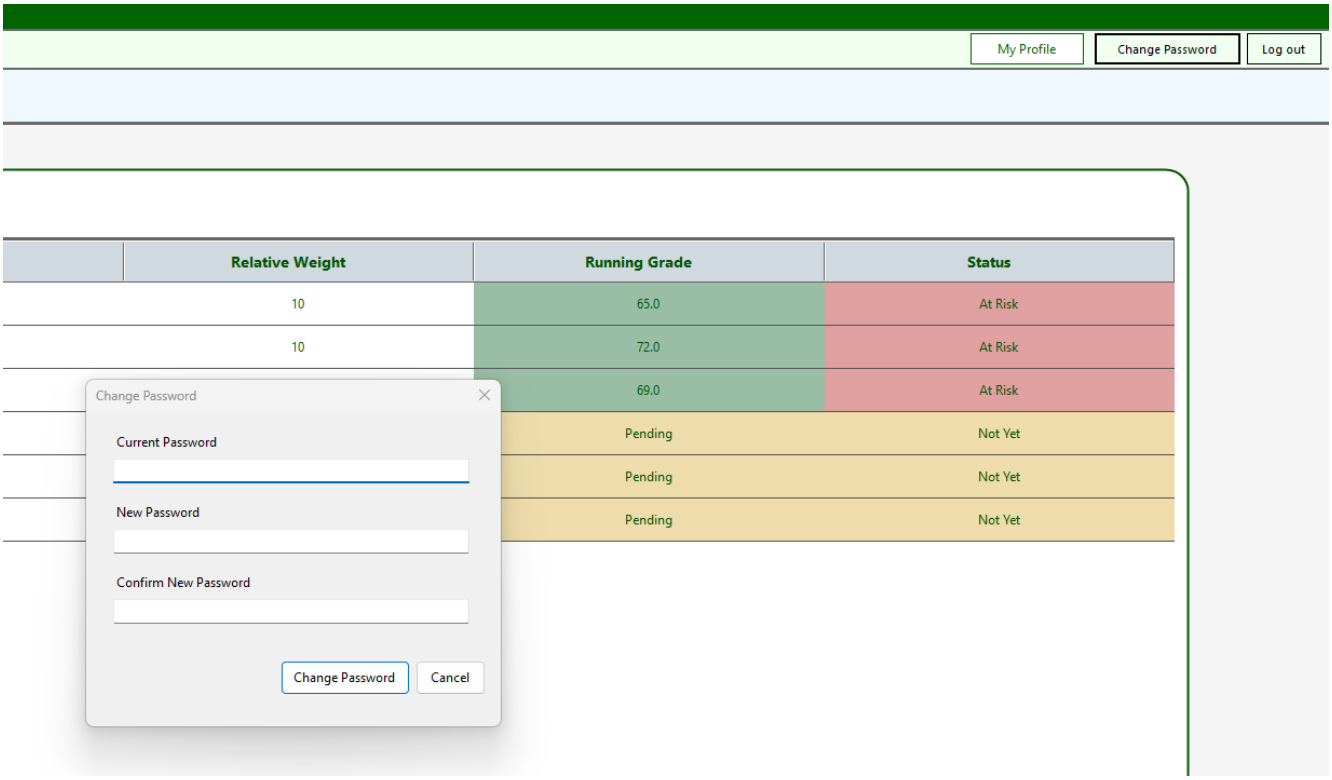
Insert screenshots with captions and short explanations.

STUDENT

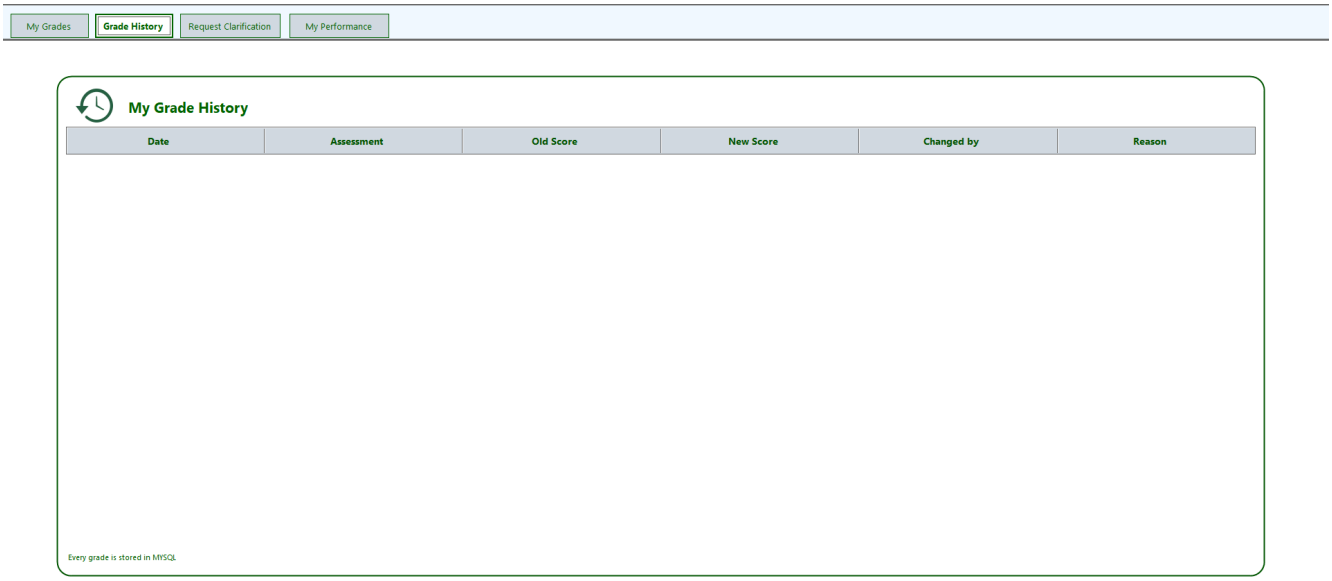


My Grades – View grades for quizzes, activities, exams, and projects. Include Midterm, Finals, and Combined views, running grades, assessment weights, and academic standing.

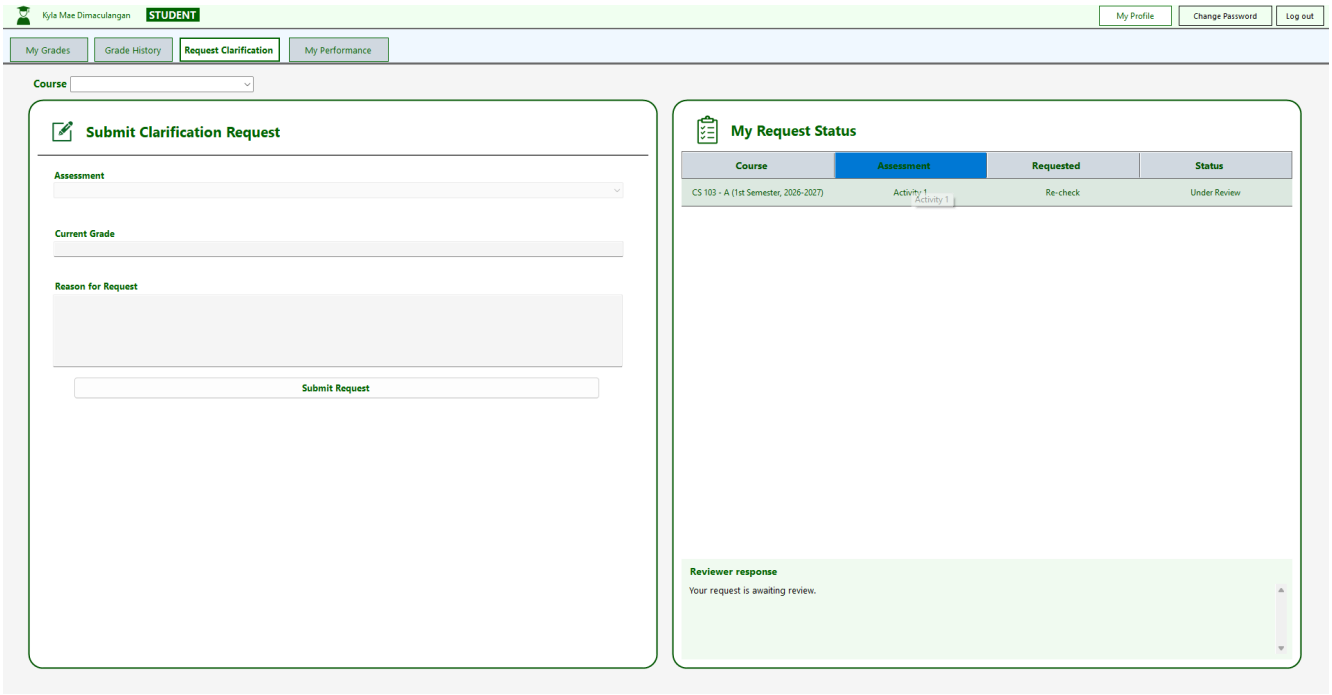




My Profile & Account Security – View student information and change the account password, with confirmation and validation.



Grade History – View changes to grades, including the date, assessment, old score, new score, person who changed the grade, and reason.

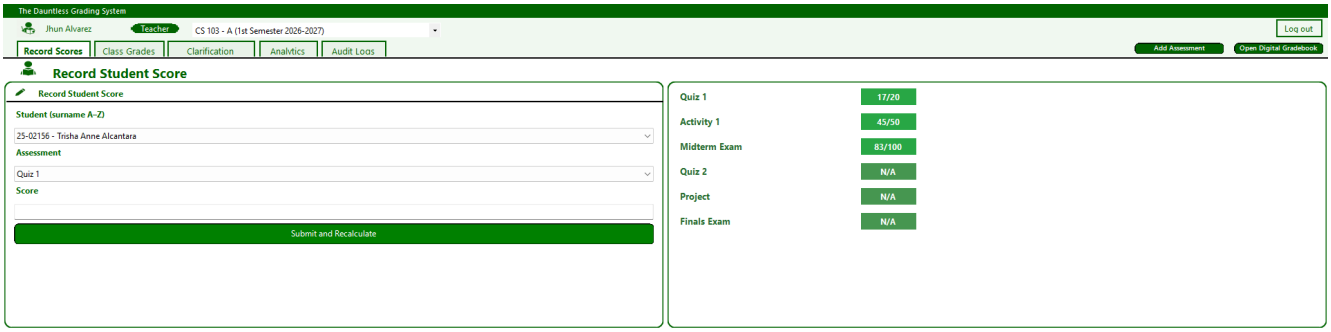


Request Clarification – Submit requests for grade rechecks, select a course and assessment, provide the current grade and reason, and track request status and reviewer responses.

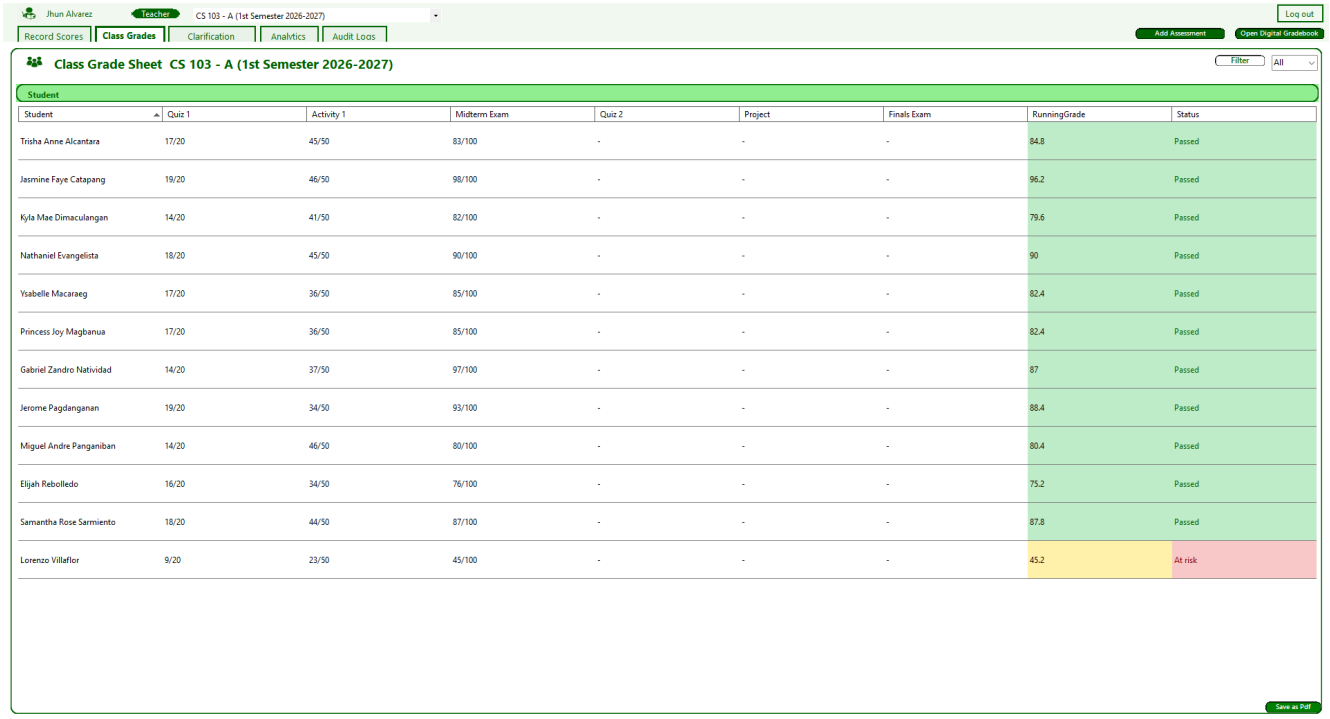


My Performance – Display the student's current average, activities passed, open requests, highest score, and performance percentages for quizzes, activities, exams, and projects.

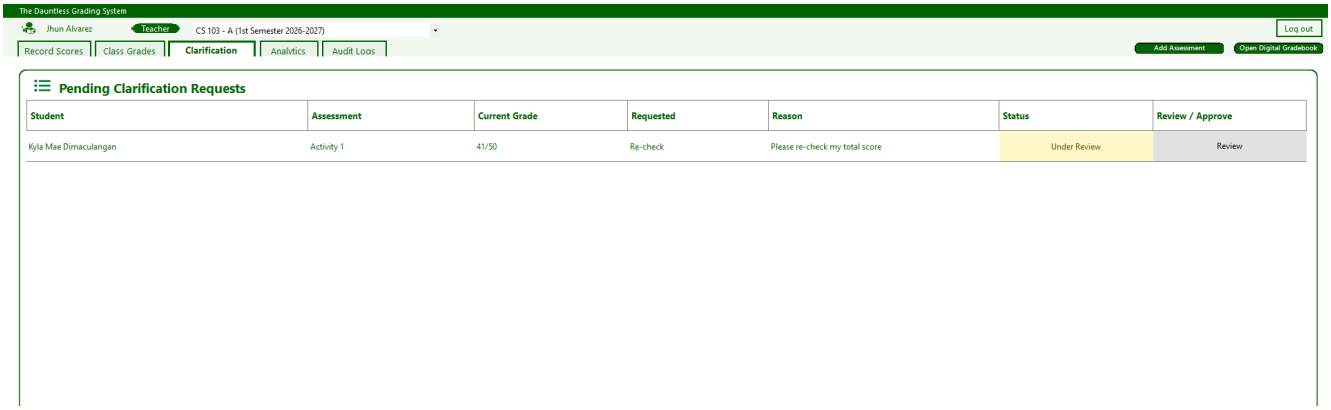
PROFESSOR



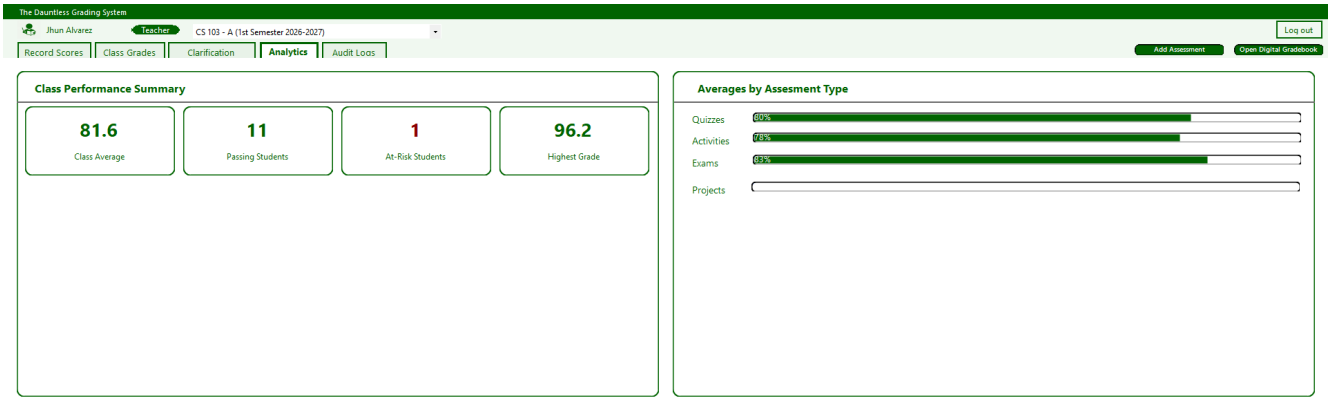
Record Student Score
Allows professors to select a student, choose an assessment, enter scores, and recalculate grades automatically.



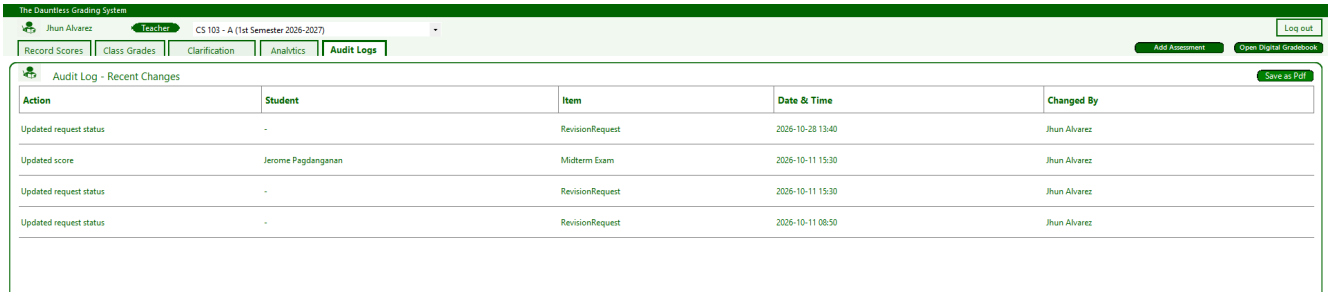
Class Grade Sheet
Displays students' assessment scores, running grades, and academic status. Professors can filter records and export the grade sheet as a PDF.



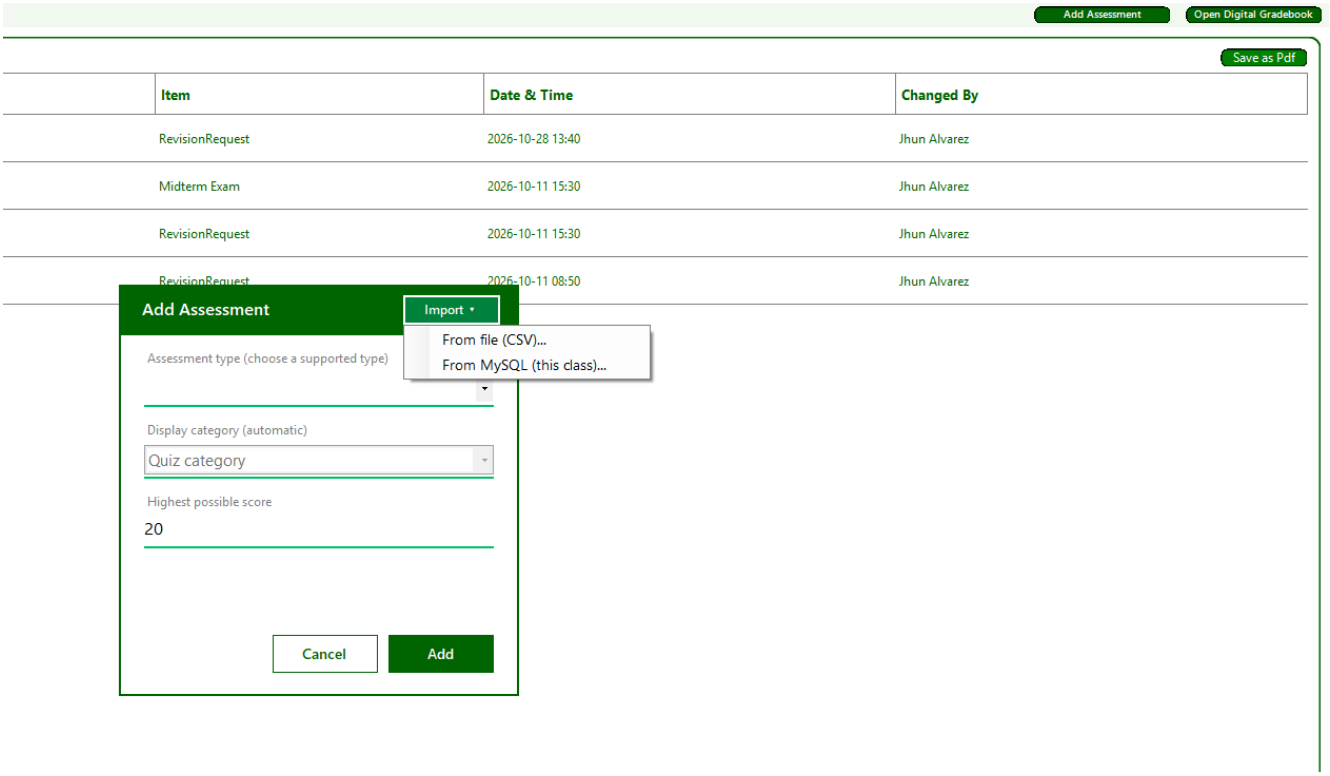
Pending Clarification Requests
Displays students' grade clarification requests, including assessment details, current grades, requested adjustments, reasons, and request statuses. Professors can review and approve requests.



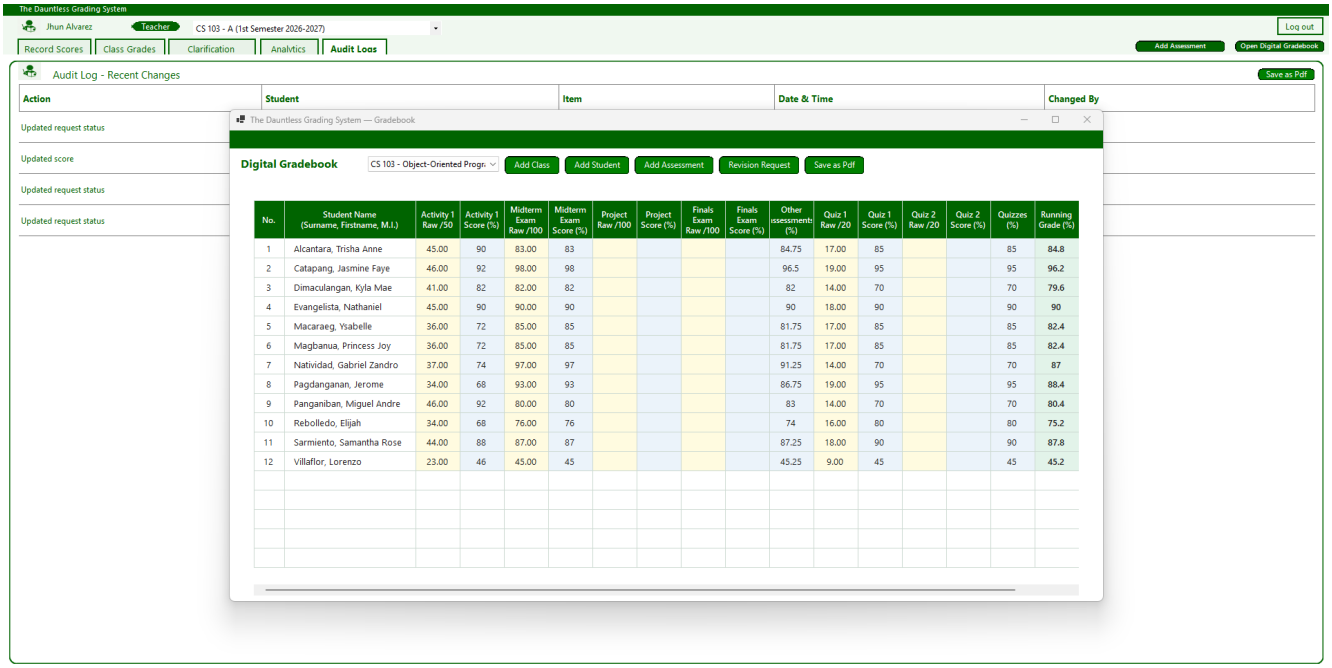
Analytics
Displays the class average, number of passing students, at-risk students, and highest grade. Progress bars summarize average performance by assessment type.



Audit Log
Records score updates and clarification request status changes, including the affected student, assessment, date and time, and responsible user. Records can be exported as a PDF.

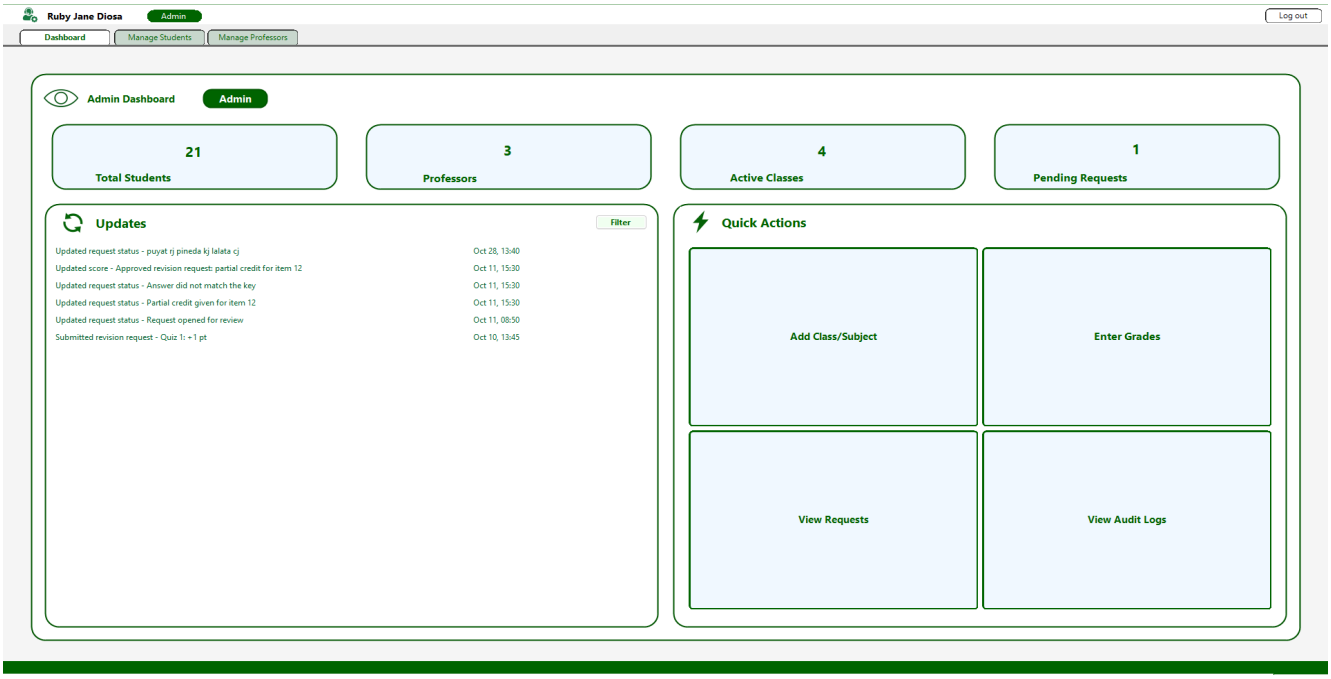


Add Assessment
Allows professors to create assessments by selecting an assessment type, display category, and highest possible score. Assessments can also be imported from CSV files or MySQL.

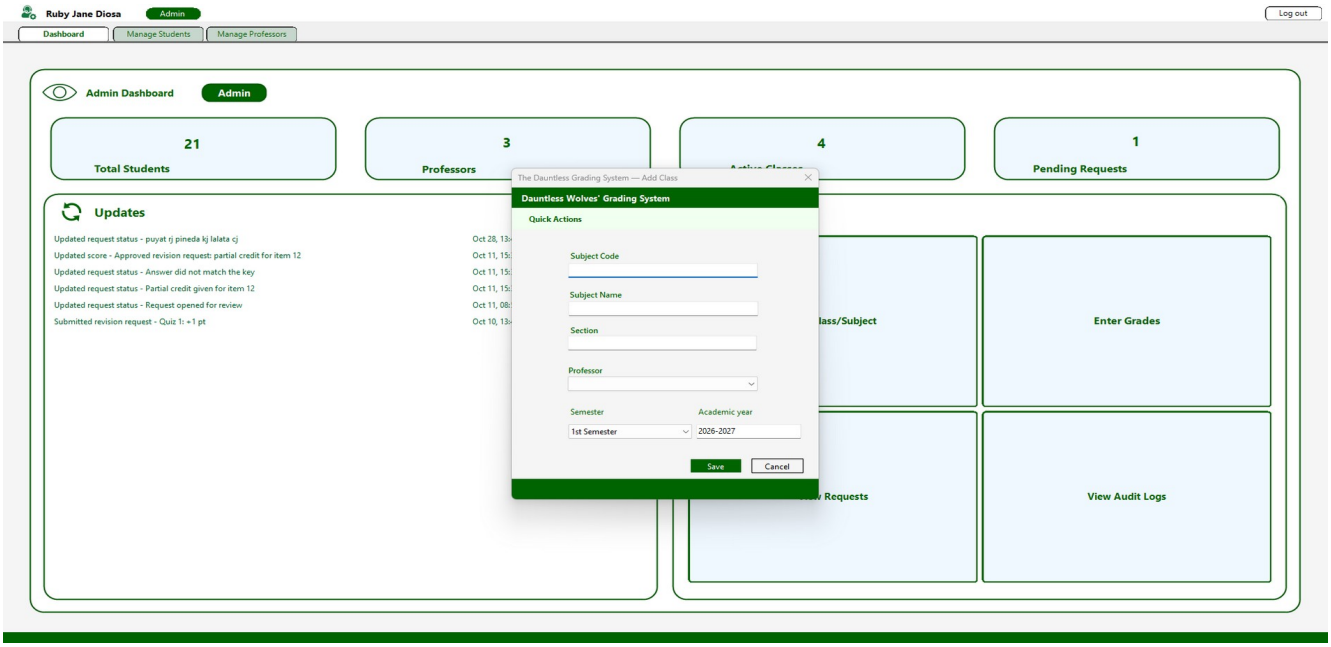


Digital Gradebook
Displays students' raw scores, percentage scores, and running grades in a spreadsheet-style layout. Professors can manage classes, students, and assessments, handle revision requests, and export the gradebook as a PDF.

ADMIN

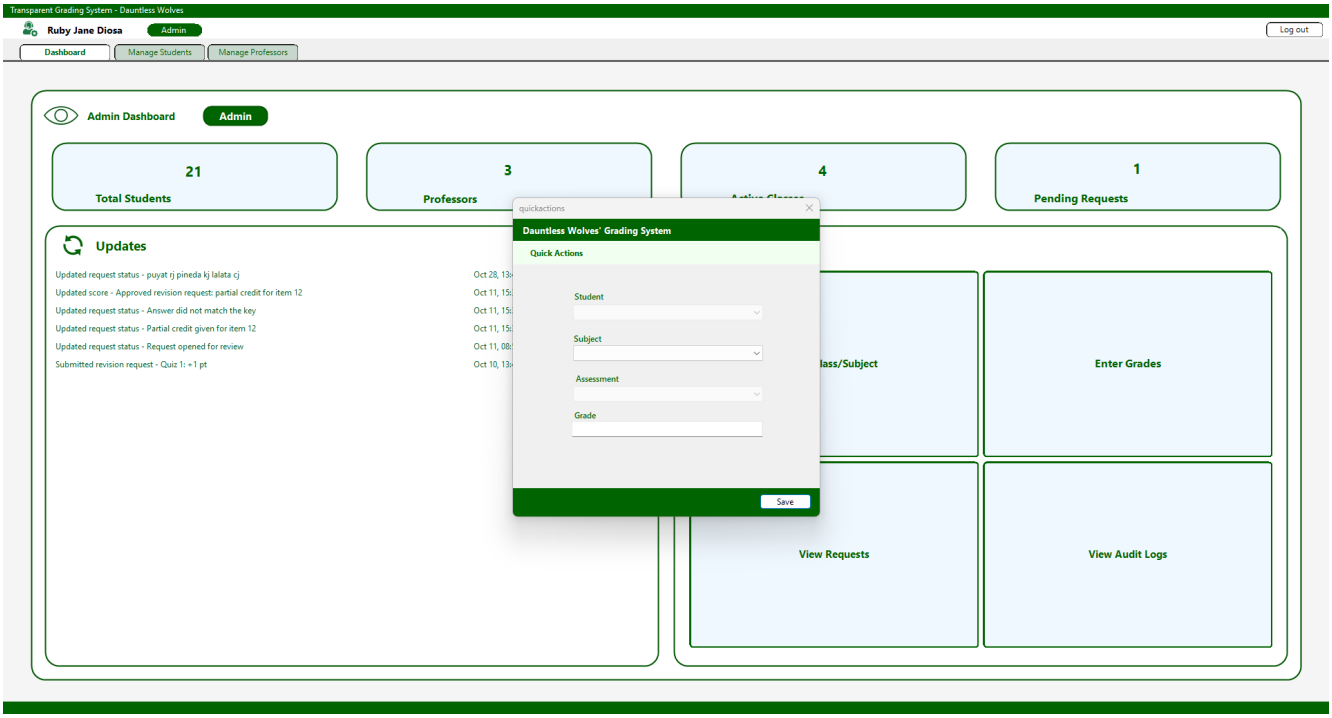


Admin Dashboard
Displays system statistics, recent updates, and quick actions for managing classes, grades, clarification requests, and audit logs.

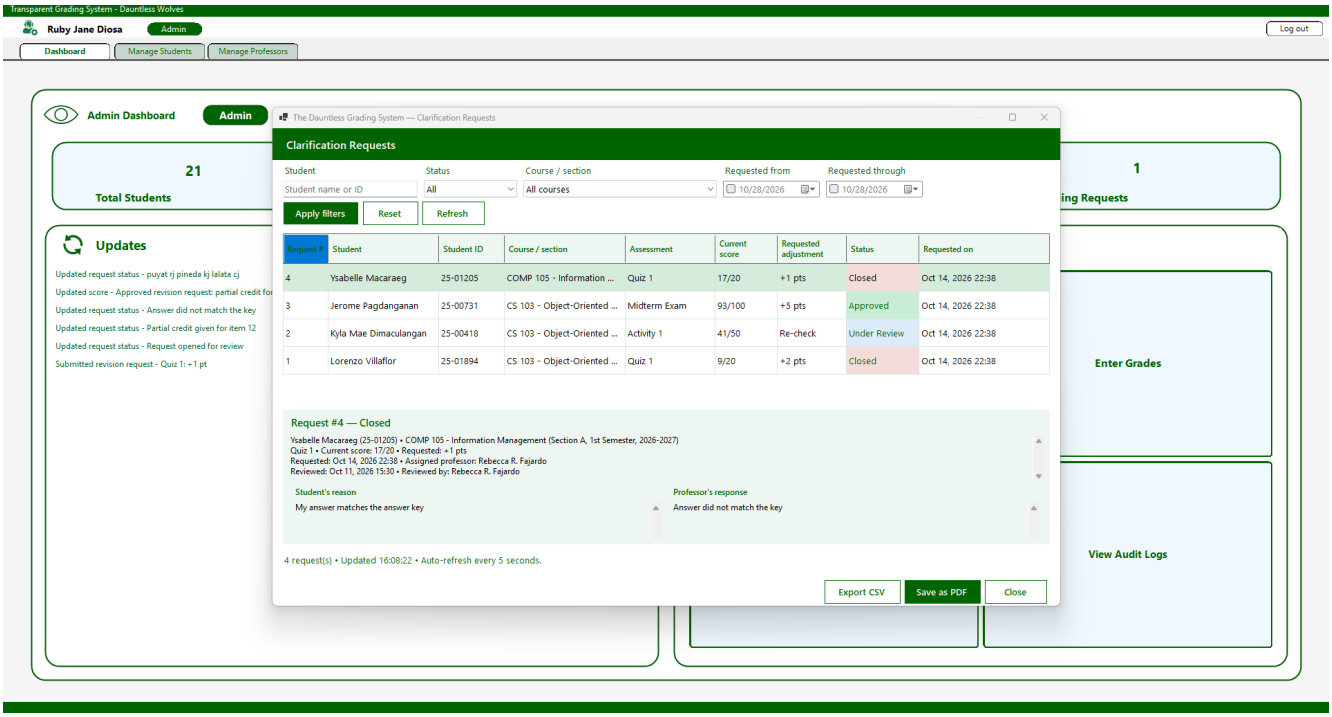


Enter Grade

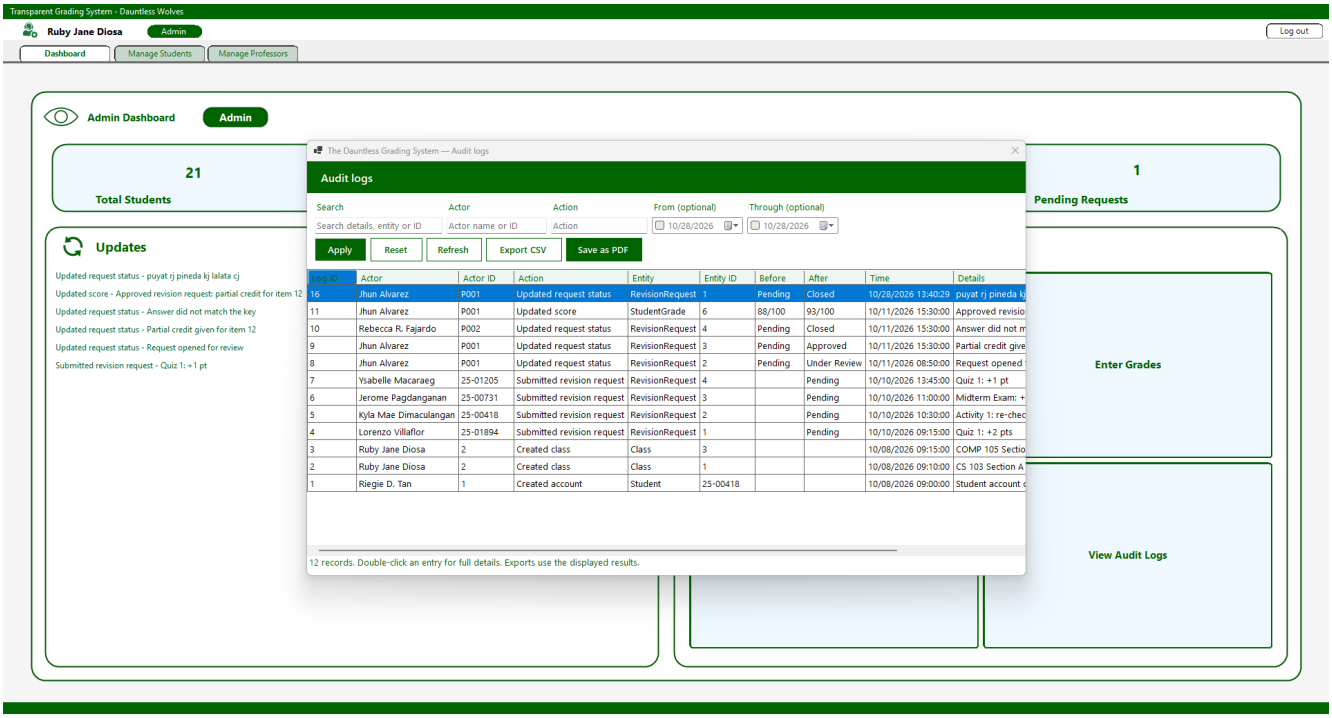
Allows administrators to insert the grade of each student specific of the matching course with the ability to choose which semester.



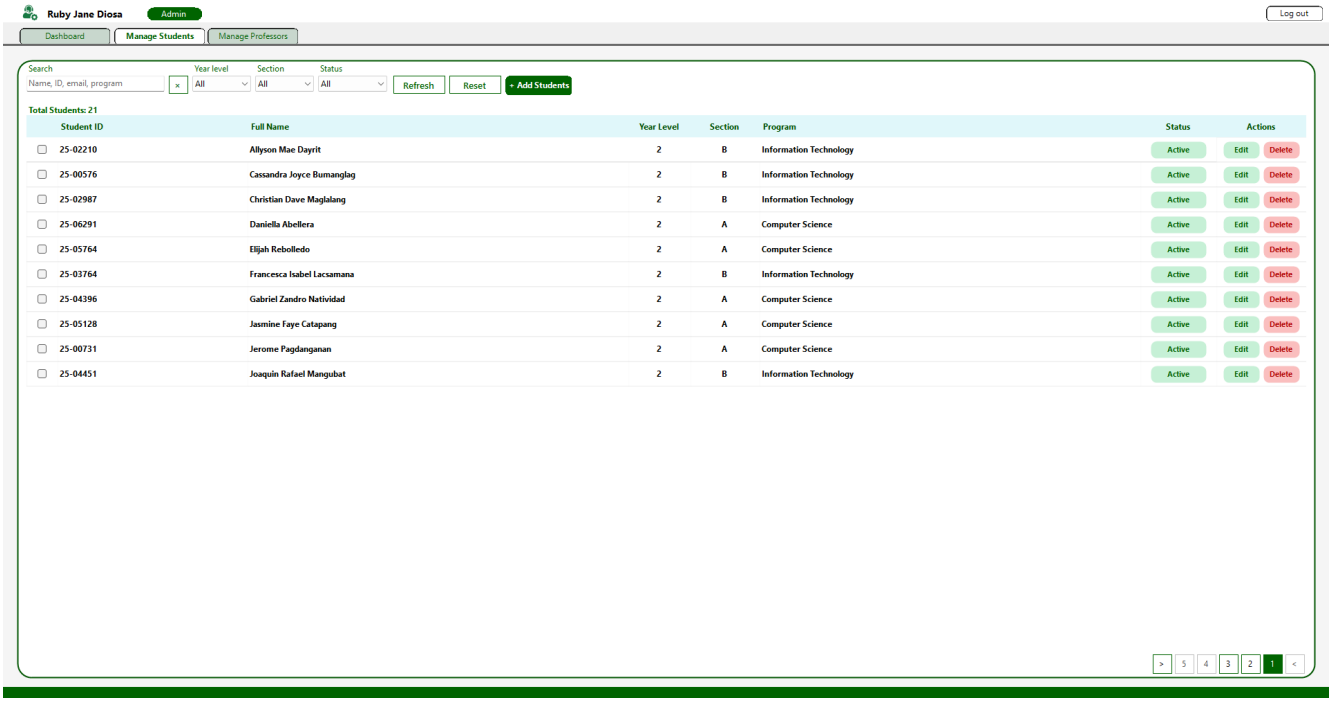
Add Class/Subject
Allows administrators to create classes by specifying subject details, sections, professors, semesters, and academic years.



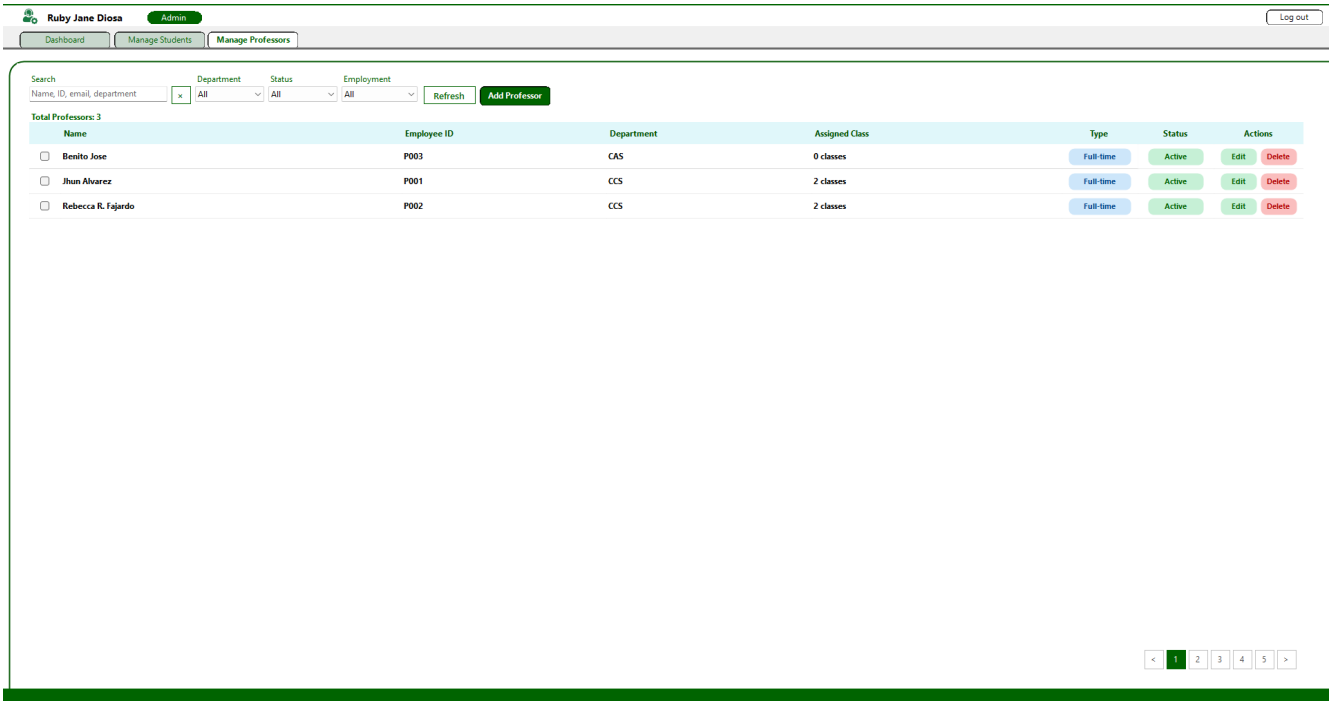
Clarification Requests
Displays students' grade clarification requests, including scores, requested adjustments, and request statuses.



Audit Logs
Records system activities, including changes made, affected records, timestamps, and details. Administrators can also export logs to CSV or PDF.



Manage Students
Allows administrators to search, filter, add, edit, and delete student records.



Manage Professors
Allows administrators to manage professor information, departments, employment types, and assigned classes.

7.8 Form/Module Inventory

Form/Module	Purpose	Tables Used	Main Functions	User Role
frmlogin	User login	Admin, Teacher, Student	Credential validation and navigation	All users
frmprof	Professor grading interface	Class, Course, Student, Enrollment, Assessment, StudentGrade, RevisionRequest, AuditLog	View class records and access grading panels	Professor
frmstudent	Student interface	Student, Enrollment, Class, Course, Assessment, StudentGrade, RevisionRequest, AuditLog	View grades and revision requests	Student
Admin	Administration interface	Admin, Teacher, Student, Course, Class, Enrollment, Assessment, StudentGrade, RevisionRequest, AuditLog	Manage users and class records	Admin

Database	Handles database access	All tables	Opens database connections and executes queries	System Module

8. User Manual

Write the manual for a first-time user. Use clear steps, screenshots, and expected results.

8.1 System Overview

The Transparent and Centralized Grading Management System for Pamantasan ng Lungsod ng Pasig (DAUNTLESS_WOLVES_PROF) is a desktop application developed in VB.NET WinForms paired with a MySQL/MariaDB database. It helps in transforming manual computation errors and lack of academic transparency by replacing manual spreadsheets with an automated, auditable, and role-based grading infrastructure.

The system serves three primary user groups:

- System Administrators: Responsible for institutional setup, account management (CRUD for students and professors), class assignments, and overall system audit monitoring.
- Faculty / Professors: Responsible for defining class assessment weightings, recording student raw scores, reviewing automated running grades, processing grade clarification requests, viewing performance analytics, and exporting official grade sheets.
- Enrolled Students: Responsible for tracking real-time academic performance across enrolled subjects, reviewing historical grade change logs, and submitting formal score revision requests with written justifications.

8.2 System Requirements

List hardware/software requirements and required database/connector components.

[Write your response here.]

8.3 Installation / Setup

Requirements: Windows 10/11, Visual Studio 2022, and MySQL/MariaDB (XAMPP).

A. Server side
    1. Create the VPN network: Radmin VPN > Network > Create New Network (name + password). Note the server's IP address.
    2. Start the database: open XAMPP and start MYSQL. Allow inbound TCP port 3306 in Windows Firewall
    3. Create the database:

        [CREATE DATABASE centralized_and_transparent_grading_system;]

    4. Import the data:

        [mysql -u root -p centralized_and_transparent_grading_system < centralized_and_transparent_grading_system.sql]

    5. Create the app user:

[CREATE USER ‘grade’@’26.%’ IDENTIFIED BY ‘<your-password>’;
GRANT SELECT, INSERT, UPDATE, DELETE ON
centralized_and_transparent_grading_system.* TO ‘gradeapp’@’26.%’;
FLUSH PRIVILEGES;}

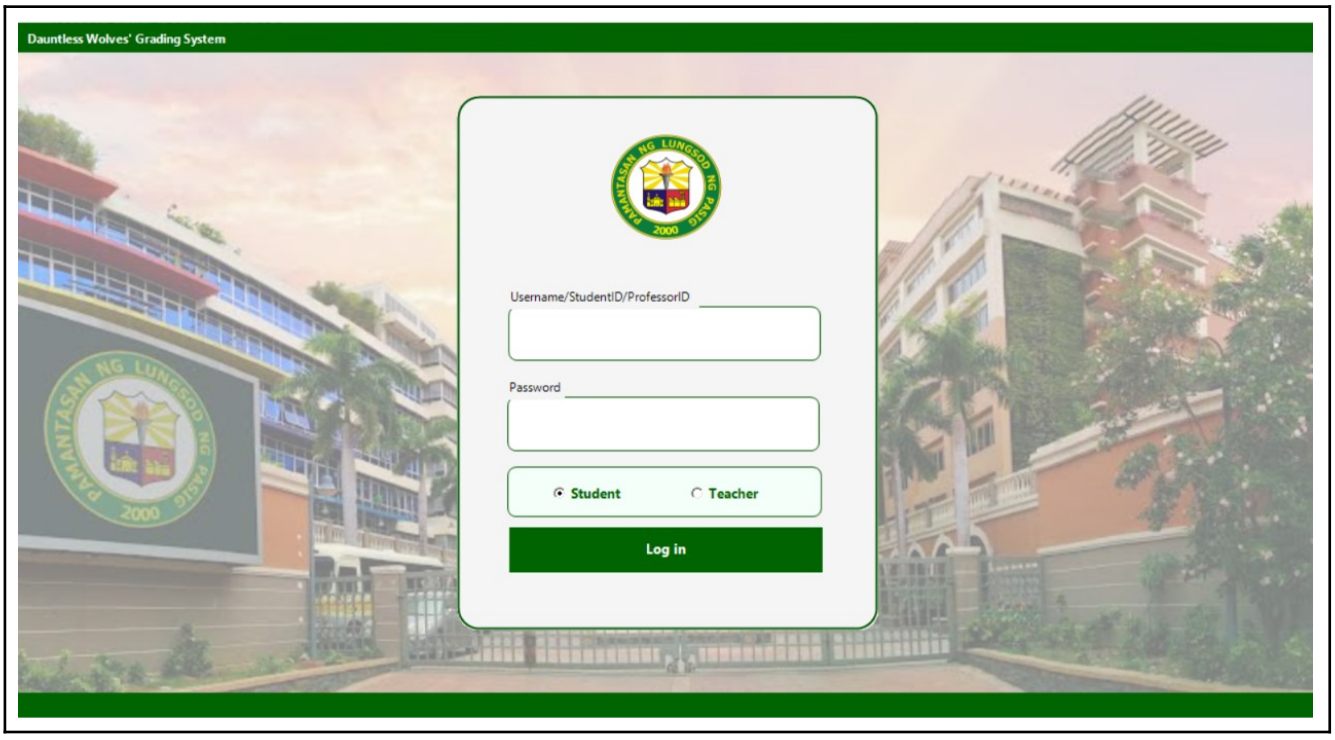
B. Client Side
   1. Join the VPN: Radmin VPN > Network > Join Network (same name + password).
   2. Install the .NET 8 Desktop Runtime
   3. Download the release zip file and unzip it
   4. Configure dbconfig.json in the same folder as the .exe:

[{
“Host”: “26.xxx.xxx.xxx”,
“Port”: 3306,
“Database”: “centralized_and_transparent_grading_system”,
“User”: “gradapp”,
“Password”: “<your-password>”
}]

Host is the server’s Radmin IP. The template is dbconfig.example.json

   5. Run the .exe and log in. “DAUNTLESS WOLVES PROF”

8.4 Login / Starting the System



1. Execute the compiled application binary (WinFormsApp2.exe).
2. The **User Login Portal** (frmlogin.vb) will open.
3. Enter your assigned **User ID / Email** into the txtEmailOrId field (e.g., 25-00418 for students, P001 for professors, or admin@plpasig.edu.ph).
4. Enter your account **Password** into the txtPassword field.

5. Click **Log In** (btnLogin).

8.5 Main Menu / Navigation

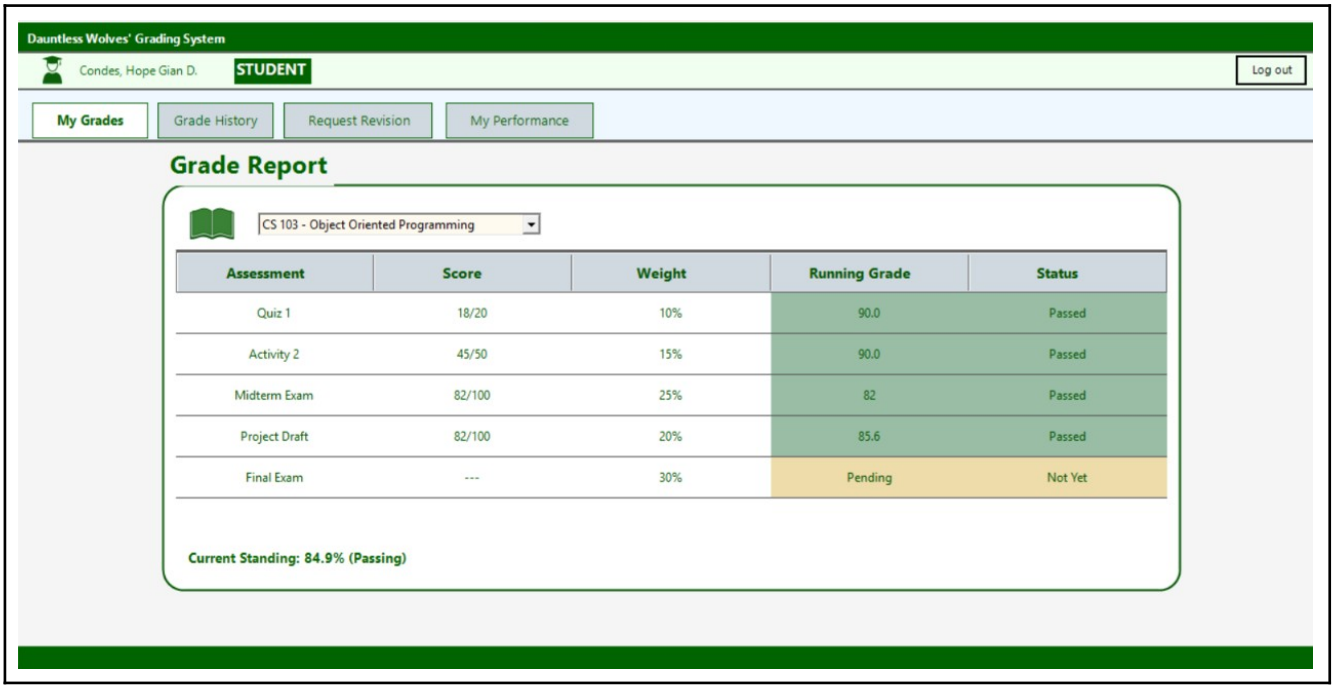


Figure 8.5.A Student Main Dashboard

1. My Grades - this form exhibits the current running grade of the student, it also shows the accomplished and pending activities of the student. Status of the assessment is also indicated here.
2. Grade History - Transparent timeline recording every score entry and modification, old vs. new values, teacher notes, and timestamps
3. Request Revision/Clarification : Interface allowing students to select a specific score, input proposed point adjustments, attach justification notes, and submit formal clarification requests.
4. My performance : It demonstrates the overall performance of the student in his/her current semester.

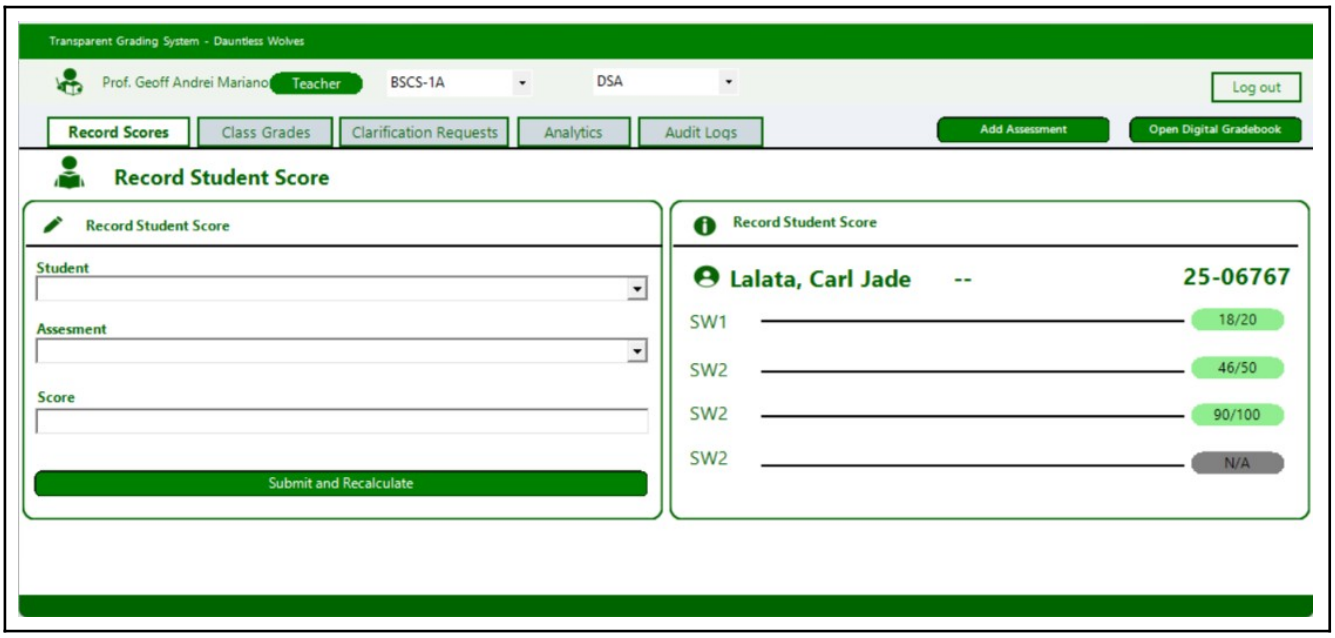


Figure 8.5.B Professors Main Dashboard

1. Shows the complete class list with automatically calculated grades, visual pass/fail indicators, options to filter by term, and the ability to export results as a PDF
2. Class Grades:Shows the complete class list with automatically calculated grades, visual pass/fail indicators, options to filter by term, and the ability to export results as a PDF
3. Revision/Clarification Requests : Queue to evaluate student dispute requests, view written justifications, and approve or reject requested score changes.
4. Class Performance Analytics: Visual graphs and progress bars showing class averages, top performers, passing distribution, and assessment breakdowns.
5. Audit Logs: Maintains a detailed, timestamped log of all grade entries, modifications, and revision approvals made by faculty to ensure data integrity.
6. Digital Gradebook: Displays a spreadsheet-style, full-screen grid of student scores across all course components, with options to export the data as a PDF.

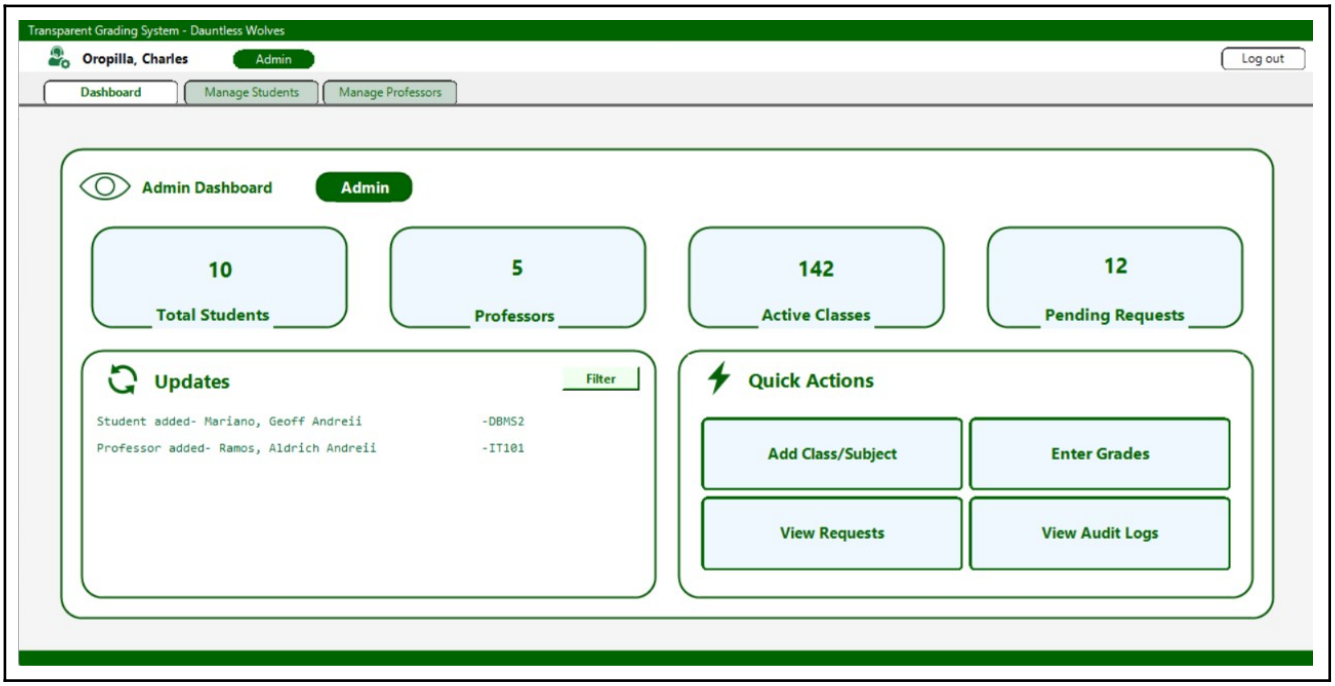


Figure 8.5.C Admin Main Dashboard

1. Dashboard Analytics: Displays quick summary counters (total students, professors, active classes, pending requests) and system health metrics.
2. Manage Students: Directory interface to add, edit, filter, or deactivate student accounts.
3. Manage Professors: Directory interface to manage faculty listings and assign academic department parameters.
4. Class & Section Offering: Module to create subject sections and link assigned professors.
5. Audit History: Comprehensive log viewer recording administrative actions across the platform.

8.6 Backup / Restore or Data Maintenance

1. Database Backup Procedure

1. Open a terminal or administrative command prompt on the database server host.
2. Run mysqldump to generate a timestamped SQL backup file:
3. Bash

```
mysqldump -u root -p centralized_and_transparent_grading_system > centralized_and_transparent_grading_system.sql
```

4. Alternatively, in phpMyAdmin, select transparent_grading_system → click Export → select Quick Export Method → click Go.

2. Database Restore Procedure

1. Create or clear the target database instance.
2. Execute the import command via terminal:
3. Bash

```
mysql -u root -p centralized_and_transparent_grading_system < centralized_and_transparent_grading_system.sql
```

4. Or import the backup `.sql` file using phpMyAdmin's **Import** tab.

9.9 Logout / Exit

1. Logging Out of an Active Session

1. Click the Logout button located on the top navigation bar of any active portal.
2. The application executes AppSession.ClearSession(), revoking user access credentials and clearing session data from memory.
3. The active portal form closes, and the system redirects back to the User Login Portal.

10. Conclusion and Recommendations

10.1 Project Conclusion

10.2 Recommendations / Future Enhancements

While the current version meets all specified project objectives, the following realistic enhancements are recommended for future iterations:

1. Cloud / Web Migration: Re-architect the client application from a WinForms desktop environment to a cloud-hosted web framework (e.g., ASP.NET Core Blazor or React with a RESTful Web API backend) to allow multi-device browser access without local software installation.
2. Automated Push & Email Notifications: Integrate an automated notification engine (using SMTP or push services) to notify students immediately via email or mobile alerts when new grades are posted or revision requests are resolved.
3. LMS and University SIS Integration: Develop API connectors to sync class rosters, enrollment records, and final term marks directly with the university Registrar's Student Information System (SIS) and Learning Management Systems (e.g., Canvas or Moodle).
4. Native Mobile Application: Build Android and iOS companion apps using MAUI or React Native to give students quick mobile access to running grades and dispute updates.
5. Predictive Performance Analytics: Incorporate machine learning models to analyze early-term assessment trends, automatically triggering early intervention alerts for students at risk of academic failure.

11. References

List references used for database concepts, programming libraries, standards, domain information, and other external materials using the citation format required by the instructor.

[Write your response here.]

12. Appendices

☐ Appendix A – Approved Concept Paper / Revision History

Problem

A. Current/Traditional Grading System

- Excel/spreadsheets are used to record grades.
- Grade files may be stored separately on individual computers.
- Students may have limited access to their current grades.
- Large amounts of grade data require manual review and organization.
- Grade revisions are directly edited in the file, potentially overwriting previous values.

B. Problems Encountered

1. **Delayed Awareness of Academic Performance** – Students may not find out early if they are failing or at risk of failing, as they may only receive their grades at the end of the semester.
2. **Accidental Errors and Overwritten Grade Information** – Teachers may accidentally enter or modify grades, and previous information can be overwritten, making it difficult to verify what was changed and potentially causing confusion or disputes.
3. **Limited Transparency in Grade Changes** – Traditional grading systems may not provide a clear record of when and how grades were modified, making it difficult to monitor and verify changes.
4. **Limited Accessibility** – Grade records may be stored on individual computers or separate files, making it difficult for students and authorized faculty to access updated grade information when needed.
5. **Difficult Manual Interpretation of Large Amounts of Grade Data** – Managing grades through spreadsheets can make it difficult and time-consuming for teachers to interpret large amounts of grade information, as they may need to manually review, organize, and calculate data to understand student and class performance.

Features to Be Implemented

1. Live/Current Grades

A real-time academic tracking feature that dynamically recalculates a student's running grade whenever an instructor logs scores for submitted activities, quizzes, or exams. This provides immediate, transparent visibility into individual task scores and overall subject standings from home. It also eliminates delayed performance feedback by removing reliance on end-of-term grade releases, students can assess whether they are passing or at risk of failing early enough in the semester to take corrective action.

2. Grade History

Grades are often vulnerable to critical issues such as data loss. To address this challenge, the concept of grade history is proposed. In traditional grading systems, once a grade is updated or overwritten, the original value is permanently lost. This can result in errors in official transcripts, disputes between students and faculty, and the disappearance of important academic records. The grade history feature prevents these problems by preserving every version of a student's grade rather than replacing it. Each entry is stored with its corresponding details, allowing both faculty and students to trace changes over time. This ensures that grades remain accurate, transparent, and recoverable, safeguarding the integrity of academic records while providing a reliable reference point for resolving issues.

3. Student & Class Performance Analytics

Instead of simply showing grades in rows and columns like a traditional Excel record, the system will organize and summarize the information into useful statistics and visual reports. The analytics feature aggregate data to display class averages, pass/fail ratios, top-performing students, and historical performance trends across different subjects and semesters. Through visual reports, charts, and summary panels, the feature will make it easier for professors to interpret grade results, visualize data, and make informed decisions without manually reviewing and calculating large amounts of data.

4. Audit Logs

To ensure that every change made to a student's grade is transparent and accountable, the proposed system will implement Audit Logs. The system will record important modifications, including who made the change, what was changed, and when it was changed, while allowing the teacher to optionally provide a reason for the change. These records will allow teachers and administrators to easily monitor activities within the system, review actions when necessary, and identify errors or unauthorized changes. This feature promotes transparency and accountability by providing a reliable record of user activities and supporting the proper management and security of academic records.

5. Grade revision requests

The system will provide a formal Grade Revision Request feature that allows students to report possible errors or discrepancies in their grades. Instead of directly changing a grade, students can submit a request indicating the specific grade, reason for the request, and supporting information. The instructor can then review the request and either approve or reject it. Approved revisions will update the student's grade while preserving the original grade and recording the change in the system. This feature provides a structured and transparent process for correcting grading errors while preventing unauthorized grade modifications and ensuring that every revision is properly documented.

Pamantasan ng Lungsod ng Pasig
Alkalde Jose St. Kapasigan, Pasig City
College of Computer Studies

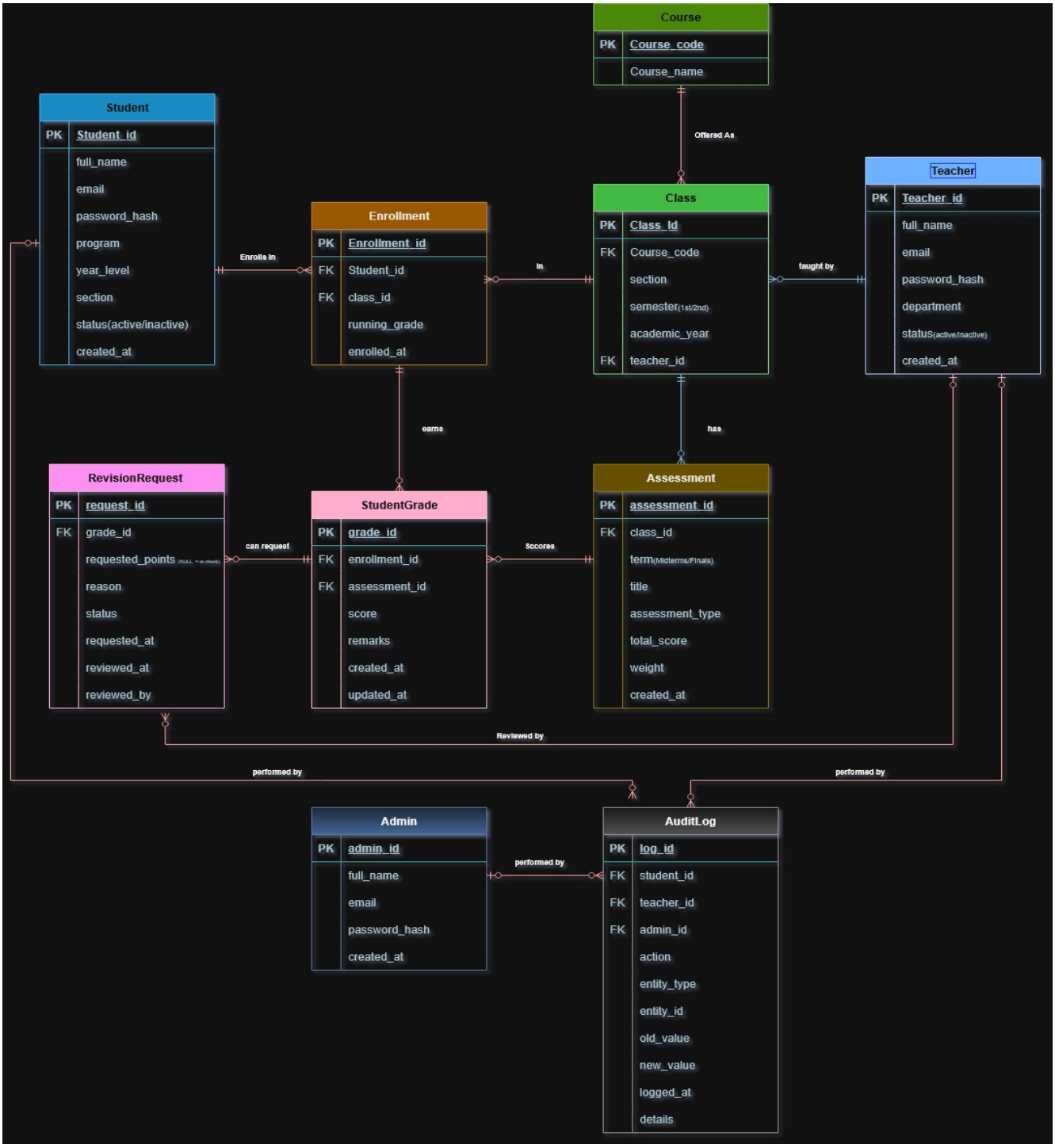
IT102 – Object Oriented Programming with Database

Activity 3
Final Project Concept
Grading System

Transparent and Centralized Grading Management System

Date: August 23, 2026
Group Name: Dauntless Wolves
Group Member: Condes, Hope Gian
Lalata, Carl Jade
Mariano, Geoff Andrei
Oropilla, Charles
Puyat, Ron Jacob

☐ Appendix B – Complete ERD



☐ Appendix C – Normalization Worksheets

Appendix D – Complete SQL Script (.sql)

Read down the left column, then down the right. SQL content is retained as supplied; long lines wrap for display.

```
create database
centralized_and_transparent_grading_system
character set utf8mb4
collate utf8mb4_unicode_ci;

use centralized_and_transparent_grading_system;

create table Student(
student_id varchar(10) primary key,
full_name varchar(100) not null,
email varchar(100) not null unique,
password_hash varchar(255) not null,
program varchar(100) not null,
year_level tinyint unsigned not null,
section varchar(20) not null,
status enum('Active','Inactive') not null default 'Active',
created_at timestamp not null default
CURRENT_TIMESTAMP);

create table Teacher(
teacher_id varchar(10) primary key,
full_name varchar(100) not null,
email varchar(100) not null unique,
password_hash varchar(255) not null,
department varchar(50) not null,
employment_type enum('Full-time','Part-time') not null
default 'Full-time',
status enum('Active','Inactive') not null default 'Active',
created_at timestamp not null default
CURRENT_TIMESTAMP);

create table Admin(
admin_id int unsigned primary key auto_increment,
full_name varchar(100) not null,
email varchar(100) not null unique,
password_hash varchar(255) not null,
created_at timestamp not null default
CURRENT_TIMESTAMP);

create table Course(
course_code varchar(20) primary key,
course_name varchar(100) not null);

create table Class(
class_id int unsigned primary key auto_increment,
course_code varchar(20) not null,
section varchar(20) not null,
semester enum('1st Semester','2nd Semester') not null,
academic_year varchar(9) not null,
teacher_id varchar(10) not null,
constraint fk_class_course
foreign key(course_code) references Course(course_code)
on update cascade on delete restrict,
constraint fk_class_teacher
foreign key(teacher_id) references Teacher(teacher_id)
on update cascade on delete restrict,
unique key uq_class (course_code, section, semester,
academic_year));

create table Enrollment(
enrollment_id int unsigned primary key auto_increment,
student_id varchar(10) not null,
class_id int unsigned not null,
running_grade decimal(5,2) null,
enrolled_at timestamp not null default
CURRENT_TIMESTAMP,
constraint fk_enrollment_student
foreign key(student_id) references Student(student_id)
on update cascade on delete restrict,
constraint fk_enrollment_class
foreign key(class_id) references Class(class_id)
on update cascade on delete restrict,
unique key uq_enrollment(student_id, class_id));

create table Assessment(
assessment_id int unsigned primary key auto_increment,
class_id int unsigned not null,
term enum('Midterm','Finals') not null,
title varchar(100) not null,
assessment_type
enum('Quiz','Activity','Exam','Project','Attendance') not null,
total_score decimal(6,2) not null,
weight decimal(5,2) not null,
created_at timestamp not null default
CURRENT_TIMESTAMP,
constraint fk_assessment_class
foreign key(class_id) references Class(class_id)
on update cascade on delete cascade,
constraint chk_weight check(weight BETWEEN 0 AND 100),
constraint chk_total_score check(total_score > 0));

create table StudentGrade(
grade_id int unsigned primary key auto_increment,
enrollment_id int unsigned not null,
assessment_id int unsigned not null,
score decimal(6,2) null,
remarks varchar(255) null,
created_at timestamp not null default
CURRENT_TIMESTAMP,
updated_at timestamp not null default
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
constraint fk_grade_enrollment
foreign key(enrollment_id) references
Enrollment(enrollment_id)
on update cascade on delete cascade,
constraint fk_grade_assessment
foreign key(assessment_id) references
Assessment(assessment_id)
on update cascade on delete cascade,
unique key uq_grade(enrollment_id, assessment_id));
```

```
create table RevisionRequest(
```

```
request_id int unsigned primary key auto_increment,
grade_id int unsigned not null,
requested_points decimal(5,2) null,
reason varchar(255) not null,
status enum('Pending','Under Review','Approved','Closed')
not null default 'Pending',
requested_at timestamp not null default
CURRENT_TIMESTAMP,
reviewed_at timestamp null,
reviewed_by varchar(10) null,
constraint fk_request_grade
foreign key(grade_id) references StudentGrade(grade_id)
on update cascade on delete cascade,
constraint fk_request_teacher
foreign key(reviewed_by) references Teacher(teacher_id)
on update cascade on delete set null);

create table AuditLog(
log_id bigint unsigned primary key auto_increment,
student_id varchar(10) null,
teacher_id varchar(10) null,
admin_id int unsigned null,
action varchar(100) not null,
entity_type varchar(50) not null,
entity_id varchar(20) not null,
old_value varchar(100) null,
new_value varchar(100) null,
logged_at timestamp not null default CURRENT_TIMESTAMP,
details varchar(500) null,
constraint fk_audit_student
foreign key(student_id) references Student(student_id)
on delete restrict,
constraint fk_audit_teacher
foreign key(teacher_id) references Teacher(teacher_id)
on delete restrict,
constraint fk_audit_admin
foreign key(admin_id) references Admin(admin_id)
on delete restrict,
constraint chk_audit_one_actor
check((student_id is not null) + (teacher_id is not null) +
(admin_id is not null) = 1),
index idx_entity (entity_type, entity_id));

(9,NULL,'P001',NULL,'Updated request
status','RevisionRequest','3','Pending','Approved','2026-10-11
07:30:00','Partial credit given for item 12'),
(10,NULL,'P002',NULL,'Updated request
status','RevisionRequest','4','Pending','Closed','2026-10-11
07:30:00','Answer did not match the key'),
(11,NULL,'P001',NULL,'Updated
score','StudentGrade','6','88/100','93/100','2026-10-11
07:30:00','Approved revision request: partial credit for item
12'),
(16,NULL,NULL,1,'Changed employment
type','Teacher','P003','Full-time','Part-time','2026-10-28
02:58:40',NULL),
(17,NULL,NULL,1,'Changed employment
type','Teacher','P003','Part-time','Full-time','2026-10-28
02:58:41',NULL),
(18,'25-00418',NULL,NULL,'Submitted revision
request','RevisionRequest','8',NULL,'Pending','2026-10-28
05:11:58','Activity 1: re-check');
```

Appendix E – Selected VB.NET Source Code / Repository Submission Details

Repository: https://github.com/CoffeeIsThere/dauntless-wolves-re-re

Reviewed branch: main | Commit: a2037141aa3bfd7da30c50218e4ef6c553eed691 | Retrieved: 2 November 2026

Project: DAUNTLESS WOLVES PROF.sln | VB.NET, .NET 8 Windows Forms, MySQL/MariaDB. The listings below are selected excerpts, not standalone programs. Helpers and supporting methods remain in the repository. Read each listing down the left column, then down the right. Visual wrapping is for documentation only.

Code E1: GradeCalculator.vb – Shared grade calculation

Source: Services/GradeCalculator.vb, lines 1–59.

Computes assessment percentages and weighted running grades. Ungraded items are excluded; recorded zeros count; no running grade is Not Yet.

```
' Pure grade math. No database, so it is easy to test.
Public Class GradeCalculator

    ' Passing mark: the ONE place to change it. Forms should use
        this constant instead of typing 75.
    Public Const PassingGrade As Double = 75.0

    Public Shared Function CalculateScorePercentage(score As
        Double, totalScore As Double) As Double
        If totalScore <= 0 Then Return 0.0
        Return Math.Round((score / totalScore) * 100, 2,
            MidpointRounding.AwayFromZero)
    End Function

    ' Kept for older callers. Statuses are worked out when
        displaying; do not store them in the database.
    Public Shared Function DetermineRemarks(percentage As
        Double) As String
        Return If(percentage >= PassingGrade, "Passed", "Needs
            Improvement")
    End Function

    ' Status of ONE assessment: Pending when not graded yet
    Public Shared Function GetScoreStatus(score As Double?,
        totalScore As Double) As String
        If Not score.HasValue Then Return "Pending"
        Return If(CalculateScorePercentage(score.Value,
            totalScore) >= PassingGrade, "Passed", "Needs
            Improvement")
    End Function

    ' Weighted running grade over graded items only:
    '   sum(score / total x weight) / sum(weight of graded
        items) x 100
    ' Nothing graded yet = Nothing (NULL), never 0.
    Public Shared Function CalculateRunningGrade(items As
        IEnumerable(Of ScoreItem)) As Double?
        Dim earned As Double = 0.0
        Dim weightUsed As Double = 0.0

        For Each it As ScoreItem In items
            If it.Score.HasValue AndAlso it.TotalScore > 0
                AndAlso it.Weight > 0 Then
                earned += (it.Score.Value / it.TotalScore) *
                it.Weight
                weightUsed += it.Weight
            End If
        Next

        If weightUsed = 0 Then Return Nothing
        Return Math.Round(earned / weightUsed * 100, 2,
            MidpointRounding.AwayFromZero)
    End Function

    Public Shared Function GetStatus(runningGrade As Double?) As
        String
        If Not runningGrade.HasValue Then Return "Not Yet"
        Return If(runningGrade.Value >= PassingGrade, "Passed",
            "At risk")
    End Function

End Class

' One graded (or not yet graded) item, used by the calculator
Public Class ScoreItem
    Public ReadOnly Property Score As Double?
    Public ReadOnly Property TotalScore As Double
    Public ReadOnly Property Weight As Double

    Public Sub New(score As Double?, totalScore As Double,
        weight As Double)
        Me.Score = score
        Me.TotalScore = totalScore
        Me.Weight = weight
    End Sub
End Class
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/GradeCalculator.vb

Code E2: AuthService.vb – Role-aware authentication

Source: Services/AuthService.vb, lines 7–51.

Looks up eligible accounts with a bound login parameter, verifies the password hash and upgrades supported older hashes after successful login.

```
Public Function VerifyLogin(login As String, rawPassword As
    String, Optional preferredRole As String = Nothing) As
    UserModel
    Dim cleanLogin As String =
        ValidationHelper.CleanInput(login)
    If cleanLogin = "" OrElse
        String.IsNullOrEmpty(rawPassword) Then Return
        Nothing

    ' Cast IDs to CHAR and fetch department/program under
        'department_or_program'
    Dim sql As String =
        "SELECT 'Admin' AS Role, CAST(admin_id AS CHAR) AS
            UserId, full_name AS FullName, email,
            password_hash, '' AS department_or_program " &
        "FROM admin WHERE email = @Login " &
        "UNION ALL " &
        "SELECT 'Teacher', CAST(teacher_id AS CHAR),
            full_name, email, password_hash, department " &
        "FROM teacher WHERE (email = @Login OR teacher_id =
            @Login) AND status = 'Active' " &
        "UNION ALL " &
        "SELECT 'Student', CAST(student_id AS CHAR),
            full_name, email, password_hash, program " &
        "FROM student WHERE (email = @Login OR student_id =
            @Login) AND status = 'Active'"

    Dim params As MySqlParameter() = {New
        MySqlParameter("@Login", cleanLogin)}
    Dim dt As DataTable = DatabaseHelper.ExecuteQuery(sql,
        params)

    For Each row As DataRow In
        dt.AsEnumerable().OrderByDescending(Function(r)
        String.Equals(r("Role").ToString(), preferredRole,
        StringComparison.OrdinalIgnoreCase))
        If preferredRole IsNot Nothing AndAlso
            row("Role").ToString() <> "Admin" AndAlso Not
            row("Role").ToString().Equals(preferredRole,
            StringComparison.OrdinalIgnoreCase) Then
            Continue For
        Dim storedHash As String =
            row("password_hash").ToString()

        If SecurityHelper.VerifyPassword(rawPassword,
            storedHash) Then
            Dim role As String = row("Role").ToString()
            Dim userId As String = row("UserId").ToString()

            ' Old SHA-256 hashes are quietly upgraded to the
            salted format after a good login
            If SecurityHelper.NeedsRehash(storedHash) Then
                UpgradeHash(role, userId, rawPassword)
            End If

            Dim deptOrProg As String = If(Not
            IsDBNull(row("department_or_program")),
            row("department_or_program").ToString(), "")

            Return New UserModel With {
                .UserId = userId,
                .FullName = row("FullName").ToString(),
                .Email = row("email").ToString(),
                .Role = role,
                .DepartmentOrProgram = deptOrProg
            }
        End If
    Next

    Return Nothing
End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/AuthService.vb

Code E3: GradeService.vb – Transactional score saving

Source: Services/GradeService.vb, lines 12–32.

Wraps the shared score-saving operation in a database transaction. RecordScoreTx performs the related authorization, validation, grade update and audit work.

```
Public Function RecordScore(enrollmentId As Integer,
    assessmentId As Integer,
                newScore As Double, teacherId As
        String,
                Optional reason As String =
        Nothing) As OperationResult
    Try
      Using conn As MySqlConnection =
        DatabaseHelper.GetConnection()
        Using tx As MySqlTransaction =
        conn.BeginTransaction()
          Try
            Dim res As OperationResult =
        RecordScoreTx(conn, tx, enrollmentId,
        assessmentId, newScore, teacherId, reason)
            If res.Success Then tx.Commit() Else
        tx.Rollback()
              Return res
            Catch
              tx.Rollback()
              Throw
            End Try
          End Using
        End Using
      Catch ex As Exception
        Console.WriteLine("[RECORD SCORE ERROR] " &
          ex.Message)
        Return OperationResult.Fail("Could not save the
          score: " & ex.Message)
      End Try
    End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/GradeService.vb

Code E4: RevisionRequestService.vb – Revision approval and score correction

Source: Services/RevisionRequestService.vb, lines 276–315.

Checks the active assigned teacher and open request, determines the corrected score, and updates the decision and grade within the caller’s transaction.

```
Private Function ApproveTx(conn As MySqlConnection, tx As
    MySqlTransaction,
                    requestId As Integer, teacherId
        As String,
                    overrideScore As Double?,
        reviewerResponse As String) As OperationResult
    If Not AccountAccess.IsActive(conn, tx, "Teacher",
        teacherId) Then Return OperationResult.Fail("Your
        professor account is inactive.")
    Dim info As RequestInfo = LoadRequest(conn, tx,
        requestId)
    Dim check As OperationResult = CheckOpenAndOwned(info,
        teacherId)
    If Not check.Success Then Return check

    Dim target As Double
    If overrideScore.HasValue Then
        target = overrideScore.Value
    ElseIf info.RequestedPoints.HasValue AndAlso
        info.CurrentScore.HasValue Then
        target = Math.Min(info.CurrentScore.Value +
            info.RequestedPoints.Value, info.TotalScore)
    Else
        Return OperationResult.Fail("NEEDSCORE")
    End If

    ' Mark the request approved (only if it is still open)
    Dim rows As Integer
    Dim updateSql As String =
        "UPDATE revisionrequest SET status = 'Approved',
            reviewed_at = NOW(), reviewed_by = @TeacherId "
            &
        "WHERE request_id = @RequestId AND status IN
            ('Pending','Under Review')"
    Using cmd As New MySqlCommand(updateSql, conn, tx)
        cmd.Parameters.AddWithValue("@TeacherId", teacherId)
        cmd.Parameters.AddWithValue("@RequestId", requestId)
        rows = cmd.ExecuteNonQuery()
    End Using
    If rows <= 0 Then Return OperationResult.Fail("The
        request could not be updated.")

    ' Apply the score through the same code path as normal
        grading (ownership, range, running grade, audit)
    Dim gradeRes As OperationResult =
        _gradeService.RecordScoreTx(conn, tx,
        info.EnrollmentId, info.AssessmentId, target,
        teacherId,
    "Approved revision request #" & requestId.ToString())
    If Not gradeRes.Success Then Return gradeRes

    ' Log the decision itself too (the score change was
        logged by RecordScoreTx)
    AuditService.LogActionTx(conn, tx, "Teacher", teacherId,
        "Updated request status", "RevisionRequest",
                    requestId.ToString(),
        info.Status, StatusApproved, "Score set to " &
        Fmt(target) & "/" & Fmt(info.TotalScore) &
        If(String.IsNullOrWhiteSpace(reviewerResponse),
        "", vbCrLf & reviewerResponse.Trim()))

    Return OperationResult.Ok("Request #" &
        requestId.ToString() & " approved. Score is now " &
        Fmt(target) & "/" & Fmt(info.TotalScore) & ".")
End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/RevisionRequestService.vb

Code E5: DatabaseHelper.vb – Parameterized database query

Source: Core/DatabaseHelper.vb, lines 75–88.

Uses the configured connection and supplied parameters to execute a query and return its rows as a DataTable.

```
Public Shared Function ExecuteQuery(sql As String,
    ParamArray params As MySqlParameter()) As DataTable
    Dim dt As New DataTable()
    Using conn As MySqlConnection = GetConnection()
        Using cmd As New MySqlCommand(sql, conn)
            If params IsNot Nothing AndAlso params.Length >
            0 Then
                cmd.Parameters.AddRange(params)
            End If
            Using adapter As New MySqlDataAdapter(cmd)
                adapter.Fill(dt)
            End Using
        End Using
    End Using
    Return dt
End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Core/DatabaseHelper.vb

Code E6: StudentProfileService.vb – Student-scoped profile summary

Source: Services/StudentProfileService.vb, lines 6–37.

Reads the logged-in student's profile, class grades and open-request count without changing academic records. Uses the shared grade calculator.

```
Public Function LoadCurrent() As StudentProfileSnapshot
    If Not AppSession.IsStudent Then Throw New
        UnauthorizedAccessException("Student access is
        required.")
    Dim studentId = AppSession.CurrentUser.UserId
    Using conn = DatabaseHelper.GetConnection()
        Using tx =
            conn.BeginTransaction(IsolationLevel.RepeatableR
            ead)
            Dim profile = ReadTable(conn, tx, "SELECT
            student_id,full_name,email,program,year_level,se
            ction,status FROM student WHERE
            student_id=@Student", studentId)
            If profile.Rows.Count <> 1 Then Throw New
            InvalidOperationException("Student profile not
            found.")
            Dim grades = ReadTable(conn, tx,
                "SELECT e.enrollment_id,c.course_code,co.cou
            rse_name,c.section,c.semester,c.academic_year,sg
            .score,a.total_score,a.weight " &
                "FROM enrollment e JOIN class c ON
            c.class_id=e.class_id JOIN course co ON
            co.course_code=c.course_code " &
                "LEFT JOIN assessment a ON
            a.class_id=c.class_id LEFT JOIN studentgrade sg
            ON sg.enrollment_id=e.enrollment_id AND
            sg.assessment_id=a.assessment_id " &
                "WHERE e.student_id=@Student ORDER BY
            c.course_code,c.academic_year,c.semester,c.secti
            on", studentId)
            Dim requests = ReadTable(conn, tx,
                "SELECT COUNT(*) AS open_count FROM
            revisionrequest r JOIN studentgrade sg ON
            sg.grade_id=r.grade_id " &
                "JOIN enrollment e ON
            e.enrollment_id=sg.enrollment_id WHERE
            e.student_id=@Student AND r.status IN
            ('Pending','Under Review')", studentId)
            Dim classes As New DataTable()
            classes.Columns.Add("Class", GetType(String))
            classes.Columns.Add("Running Grade (%)",
            GetType(Double))
            classes.Columns.Add("Standing", GetType(String))
            For Each group In
            grades.AsEnumerable().GroupBy(Function(row)
            CInt(row("enrollment_id")))
                Dim first = group.First()
                Dim items = group.Where(Function(row) Not
            IsDBNull(row("total_score")) AndAlso Not
            IsDBNull(row("weight"))).
                    Select(Function(row) New
            ScoreItem(If(IsDBNull(row("score")),
            CType(Nothing, Double?), CDbl(row("score"))),
            CDbl(row("total_score")), CDbl(row("weight"))))
                Dim grade =
            GradeCalculator.CalculateRunningGrade(items)
                Dim label = first("course_code").ToString()
            & " — " & first("course_name").ToString() & " |
            " & first("section").ToString() & " | " &
            first("semester").ToString() & " | " &
            first("academic_year").ToString()
                classes.Rows.Add(label, If(grade.HasValue,
            CType(grade.GetValueOrDefault(), Object),
            DBNull.Value), GradeCalculator.GetStatus(grade))
            Next
            tx.Commit()
            Return New StudentProfileSnapshot With {.Profile
            = profile.Rows(0), .Classes = classes,
            .OpenRequests =
            CInt(requests.Rows(0)("open_count"))}
        End Using
    End Using
End Function
```

Full source: https://github.com/CoffeeIsThere/dauntless-wolves-re-re/blob/a2037141aa3bfd7da30c50218e4ef6c553eed691/Services/StudentProfileService.vb

☐ Appendix F – Additional Test Evidence

☐ Appendix G – Additional User Manual Screenshots

☐ Appendix H – Other Instructor-Required Evidence

Final Submission Checklist

☐ Concept Paper is complete and approved/revised.

☐ Business Rules are clear and reflected in the ERD.

☐ ERD matches the final normalized tables.

☐ Normalization process shows 1NF, 2NF, and 3NF—not only the final answer.

☐ Data Dictionary matches the MySQL database.

☐ MySQL database contains correct constraints and sufficient sample data.

☐ VB.NET system is connected to the submitted MySQL database.

☐ CRUD, validation, and required project functions are working.

☐ Test Cases include expected and actual results with evidence.

☐ User Manual is understandable to a first-time user.

☐ All screenshots are readable and captioned.

☐ All files use the required naming convention and are submitted in the required format.

REQUIRED Electronic Submission Package

Recommended folder structure: Folder name: Group # and Title

```
01_Documentation/
  Final_Project_Documentation.docx
  Final_Project_Documentation.pdf
02_Database/
  database.sql
  sample_data.sql
03_VBNet_System/
  [Visual Studio solution/project files]
04_Test_Evidence/
  [screenshots and test files]
05_User_Manual/
  User_Manual.pdf

06_Grade/
  Indi_score.xlsx
```